

AI-DLC COWORKER

User Guide

Responsible Use, Onboarding, Delivery Workflows, and Troubleshooting

Internal Use Only

Version 1.1 | Updated 16th September 2026

Table of Contents

AI DLC Coworker — User Guide & Responsible Use Documentation	4
Table of Contents	4
1. Responsible Use of AI DLC Coworker (Read First)	5
1.1 Core Principles	5
1.2 Appropriate Use.....	5
1.3 Prohibited Use.....	6
2. What Is AI DLC Coworker?	6
What It Brings Together	6
What AI DLC Coworker Does Not Do	7
3. AI Capabilities, Limitations & Known Risks	7
3.1 What the AI Does Well	7
3.2 Known Limitations	7
3.3 Why Human Review Is Mandatory	8
4. Your Responsibilities as a User (Human-in-the-Loop).....	8
Your Obligations	8
5. Data Handling & Privacy Rules	9
5.1 Do NOT Input the Following	9
5.2 What You CAN Use.....	9
5.3 Platform Safeguards.....	9
6. Who Uses It?.....	10
7. Team Onboarding	10
Getting Started	10
Onboarding Review	11
After Approval	11
Setup Progress Checklist	11
Next Steps for the Team Owner	11
8. Secure Group Ownership	12
Recommended Ownership Model.....	12
How to Create a Secure Group	12
9. Access Roles	13
Practical Recommendation	13

10. Access Levels In the App	13
Viewer Access	13
Editor Access	14
11. Signing In.....	14
12. First-Time Team Setup	14
13. Step 1: Set Up Your Context	15
What Is Context?	15
How to Set Up Context	16
14. Step 2: Explore With Playground Agent	16
What It Does	16
How To Use It	16
Example Prompts.....	17
15. Step 3: Set Up Your Workspace	17
What Is Workspace Setup?	17
Key Concepts	17
How To Set Up Your Workspace.....	18
16. Step 4: Generate Requirements	18
What It Does	18
How To Use It	19
Example Prompts.....	19
17. Step 5: Break Down Features With Product Agent	19
What It Does	19
How To Use It	20
18. Step 6: Implement With CodeNow Agent	20
What It Does	20
Default Teams: Delegations to GitHub Copilot Cloud Agent	21
ServiceNow Teams: CodeNow Studio.....	22
How To Use CodeNow	23
What CodeNow Can Generate	23
19. Step 7: Generate Tests With QA Agent.....	23
What It Does	23
How To Use It	23
20. Analytics Dashboard.....	24

What You Can See.....	24
Cost Calculation.....	24
21. TCO Dashboard	25
Prerequisites	25
What the TCO Dashboard Shows	25
22. Settings Overview.....	26
Connected System Configuration	26
Rally Configuration.....	26
Git Repository Configuration	26
Context and Graphify Configuration	26
Prompt and Skill Configuration.....	27
23. Best Practices	27
24. Reporting Issues, Feedback & Escalation	28
What to Do.....	28
Where to Report.....	28
25. Troubleshooting.....	29
I cannot sign in or I see an unauthorized page	29
I signed in but do not see my team in the Team Selector	29
I can see the team but cannot edit settings or configurations.....	29
I cannot start the pilot	29
An agent says no connected-system configuration is available	30
Rally actions are unavailable or features are not loading	30
GitHub integration or Delegations are not available	30
Delegations are not progressing	30
Generated output looks off, biased, or is not relevant to my context	30
The repository list in Git settings only shows 250 repositories	30
TCO Dashboard shows no data	31
26. Glossary	31

 **App URL:** <https://aidlc-coworker.optum.com/>

Document Owner: AI-DLC Coworker Platform Team

Classification: Internal Use Only

Document version	1.0 Draft
Last updated	19 August 2026
Last validated	19 August 2026 - content review; production capability validation pending
Document owner	AI-DLC Coworker Platform Team
Validation owners	Product, Engineering, Security, Privacy, and Responsible AI
Review cadence	At each material product release or policy change

⚠ Responsible AI Notice AI-DLC Coworker is an AI-assisted productivity tool that generates **advisory draft outputs only**. It does **not** make autonomous decisions and does **not** deploy any change to a downstream system. **You, the human user, remain fully accountable** for reviewing, validating, editing, and approving every output before it is used. This guide includes mandatory Responsible AI guidance that all users must read before using the platform.

Table of Contents

1. Responsible Use of AI-DLC Coworker (Read First)
2. What Is AI-DLC Coworker?
3. [AI Capabilities, Limitations & Known Risks](#)
4. [Your Responsibilities as a User \(Human-in-the-Loop\)](#)
5. [Data Handling & Privacy Rules](#)
6. [Who Uses It?](#)
7. [Team Onboarding](#)
8. [Secure Group Ownership](#)
9. [Access Roles](#)
10. [Access Levels In the App](#) Access Levels within the App
11. [Signing In](#)
12. [First-Time Team Setup](#)
13. [Step 1: Set Up Your Context](#)
14. [Step 2: Explore With Playground Agent](#)
15. [Step 3: Set Up Your Workspace](#)
16. [Step 4: Generate Requirements](#)
17. [Step 5: Break Down Features With Product Agent](#)
18. [Step 6: Implement With CodeNow Agent](#)
19. [Step 7: Generate Tests With QA Agent](#)
20. [Analytics Dashboard](#)
21. [TCO Dashboard](#)
22. [Settings Overview](#)
23. [Best Practices](#)
24. [Reporting Issues, Feedback & Escalation](#)

25. Known Issues and Limitations

The items below reflect confirmed user-reported limitations. Known issues are separate from user-correctable troubleshooting and may require Platform Team follow-up.

- **GitHub attribution:** Repository activity created by a delegated coding agent may display the agent rather than the initiating user. Verify repository attribution and audit requirements for the workflow.
- **GitHub PAT operations:** Some PAT-based GitHub operations have been reported as unsuccessful. Capture the affected operation and report it to the Platform Team.
- **Testing sessions:** Older sessions may appear selectable but fail because the session has expired. Create a new session and report the behavior if it persists.
- **Blank generated test cases:** A successful generation status may occasionally be followed by blank test-case content in review. Do not approve an empty result; regenerate or report the issue.
- **Large Requirement Agent context:** Large inputs may be difficult to enter because of scrolling behavior. Use supported source documents where practical and report blocking UI behavior.

25. [Troubleshooting](#)

26. [Glossary](#)

1. Responsible Use of AI-DLC Coworker (Read First)

AI-DLC Coworker is intended to be used in accordance with applicable UHG Responsible AI, privacy, security, and data-handling requirements. Users must follow the approved use conditions communicated by the Platform Team and all applicable enterprise policies. Before using any agent, every user must understand and follow the principles below.

1.1 Core Principles

Principle	What It Means for You
Human Accountability	The AI assists; you decide . You are accountable for any output you accept, edit, approve, push to Rally, merge, or release.
AI Generates Drafts Only	Every output (requirement, story, code, test, chat answer) is a reviewable draft , never a final or authoritative decision.
No Autonomous Action	The platform does not autonomously merge code or deploy changes to production. Actions that create or update artifacts in connected systems, such as Rally items, GitHub issues, pull requests, or test executions, require an authorized user action or an applicable human approval step.
Exercise Independent Judgement	You are never required to accept AI output. You may always override, edit, reject, or ignore any suggestion.
Transparency	The UI clearly labels AI-generated content and records the human decisions taken on it via the Orchestrator timeline.

Principle	What It Means for You
Verify Before You Trust	AI output can be incorrect, incomplete, outdated, or fabricated ("hallucinated"). Always validate against authoritative sources and your own expertise.
Protect Data	Never input PHI, PII, member, patient, provider, payment, credentials, or other Protected/regulated data into any agent.

1.2 Appropriate Use

AI-DLC Coworker is approved **only** for internal software-delivery productivity tasks, including:

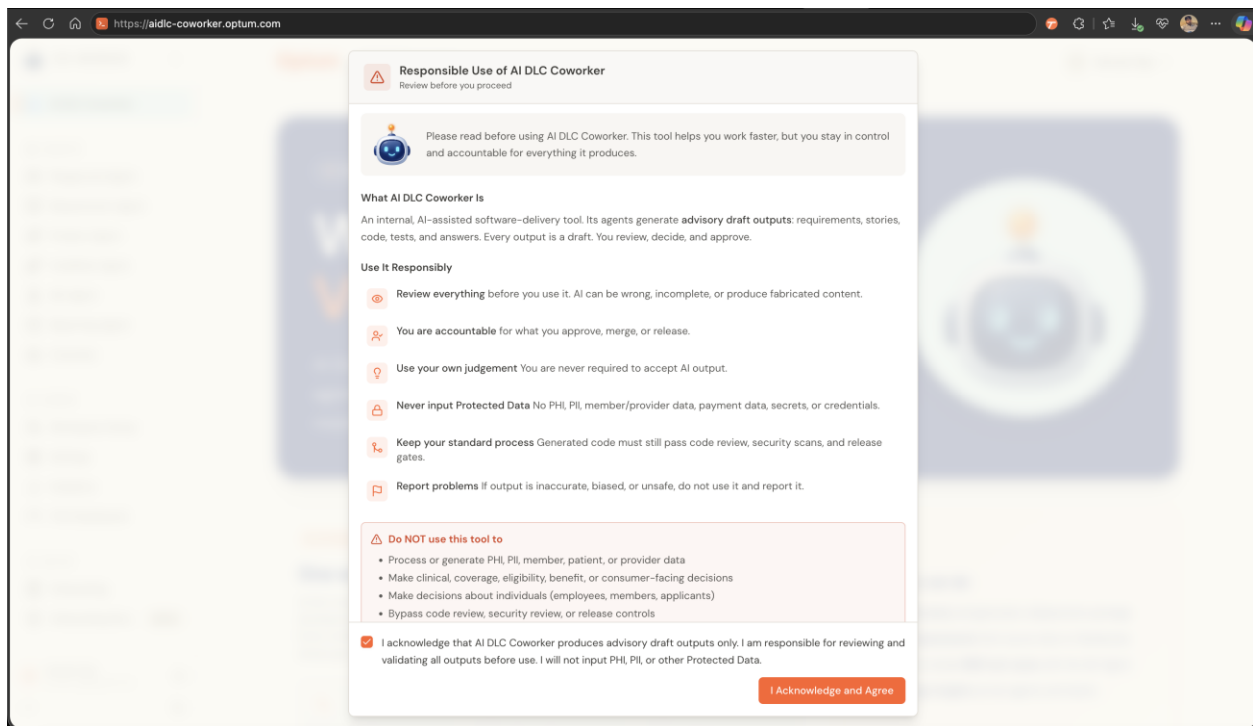
- Understanding existing internal systems and codebases
- Drafting requirements, user stories, and tasks
- Generating reviewable code, configuration, and scripts
- Generating BDD test scenarios
- Onboarding and contextual Q&A about connected internal systems

1.3 Prohibited Use

You must **not** use AI-DLC Coworker to:

- Process or generate **PHI, PII, member, patient, or provider data**
 - Make or automate **consumer-facing, clinical, coverage, eligibility, or benefit decisions**
 - Generate outputs used to evaluate, rank, or make decisions about **individuals** (employees, members, applicants, etc.)
 - Bypass your team's standard code review, security review, validation, or release processes
 - Input third-party confidential information you are not authorized to share
 - Use outputs for any purpose outside internal engineering and delivery work
- If you believe a use case may fall outside approved use, **stop and contact the Platform Team** before proceeding.

[Responsible Use acknowledgement / banner shown to users](#)



2. What Is AI-DLC Coworker?

AI-DLC Coworker is an AI-powered, team-scoped delivery workspace that brings together a set of specialized AI agents to accelerate the full software development lifecycle. It covers requirement generation, product breakdown, code implementation, testing, and workflow visibility in one connected platform.

The platform is built around a simple idea: **intent should travel with the work**. A business idea, feature request, or source document captured at the start of delivery should remain traceable through every downstream artifact, requirements, stories, code, and tests without being restated or lost at handoffs.

Each team gets its own **isolated workspace**. Prompts, configurations, context documents, and conversations are all scoped to the team. Agents operate only on the context your team provides. They do not have access to production systems or sensitive data unless you explicitly configure and upload that context.

Underlying technology: AI-DLC Coworker uses approved large language models through enterprise-authorized model access channels. The model used may vary by agent, capability, configuration, and approved model profile. Data handling, retention, and model-provider controls are governed by the applicable enterprise platform and model-access requirements.

What It Brings Together

Capability	Description
Context and Knowledge	Upload documents and architecture diagrams, and run Graphify to build a knowledge graph of your codebase
Requirements Work	Generate structured requirement documents from source documents, discussions, and rough notes
Product Breakdown	Decompose Rally features into reviewable user stories and tasks
Coding Assistance	Generate reviewable implementation packages or delegate repository work to GitHub Copilot cloud agent
Quality Support	Generate and review BDD test scenarios
Workflow Visibility	Track initiatives, features, assignments, timelines, and cross-agent activity
Cost and Value Tracking	Track token consumption, cost per feature, and compare productivity before and after AI-DLC adoption

What AI-DLC Coworker Does Not Do

AI-DLC Coworker does not autonomously merge code or deploy changes to production. Agents analyze approved context and generate reviewable outputs, including requirement documents, code packages, test scenarios, and pull requests. Actions that create or update artifacts in connected systems require an authorized user action or an applicable human approval step.

3. AI Capabilities, Limitations & Known Risks

To use AI-DLC Coworker responsibly, you must understand what the AI can and cannot reliably do.

3.1 What the AI Does Well

- Summarizing and structuring scattered source material into draft requirements
- Decomposing features into candidate stories and tasks
- Generating boilerplate, reference implementations, and reviewable code
- Producing first-draft BDD test scenarios
- Answering contextual questions about connected systems and uploaded context

3.2 Known Limitations

Limitation	What to Watch For
Hallucination	The AI can generate plausible-sounding but incorrect or fabricated content, including non-existent APIs, fields, or logic.
Incomplete Context	The AI only knows what is in your uploaded context, knowledge graph, and prompt. Missing context can result in less relevant or incomplete output.
Security & Quality Gaps	Generated code may contain bugs, inefficiencies, or security weaknesses. It must pass your standard reviews and scans.
Inconsistency	The same prompt may produce different results across runs or prompt versions.

Limitation	What to Watch For
No Real-Time System State	The AI does not have live access to production system state unless explicitly configured as context.

3.3 Why Human Review Is Mandatory

Because of these limitations, **no AI output may be used downstream without human review**. Every agent in the platform enforces an explicit review/approve step by design (see Section 4).

Remember: Accepting AI output without review does not transfer responsibility to the AI. **You remain accountable** for the artifacts you approve.

4. Your Responsibilities as a User (Human-in-the-Loop)

Human review is a **core design principle** of AI-DLC Coworker. Each agent produces draft artifacts that **must** be reviewed and approved by a human before they are used downstream.

Agent	Your Required Human Action
Requirement Agent	Review the generated requirement candidate (with section-level diff). Approve, edit, or reject before it is pushed to Rally.
Product Agent	Review each generated story and task in the review session. Approve, reject, or edit per item before anything is written to Rally.
CodeNow Agent	Review generated files / pull requests. Run your team's normal code review, validation, security scanning, and release process. Delegated work returns as a PR that an engineer must review and merge .
QA Agent	Review each BDD scenario. Approve, reject, edit, or regenerate before final approval.
Playground Agent	Read-only. Treat answers as context only , not as authoritative fact. Verify before acting.

Your Obligations

27. **Review every output** before using, approving, or merging it.
28. **Validate accuracy** against authoritative sources, standards, and your own expertise.
29. **Apply security and quality gates** — do not let AI output bypass standard reviews.
30. **Exercise independent judgement** — override or reject anything that is wrong, risky, or non-compliant.
31. **Maintain traceability** — let the Orchestrator timeline record decisions; do not work around it.
32. **Report problems** — flag inaccurate, biased, unsafe, or unexpected output (see Section 24).

The Orchestrator records each agent's output and the human decisions taken on it, preserving traceability and accountability across the full delivery flow.

5. Data Handling & Privacy Rules

AI-DLC Coworker is **not approved** to process Protected Data. Following these rules is mandatory.

5.1 Do NOT Input the Following

- **PHI** (Protected Health Information)
- **PII** (Personally Identifiable Information)
- **Member, patient, or provider data**
- **Cardholder / payment data**
- **Credentials, secrets, API keys, tokens** (except where the platform explicitly provides secure credential fields, e.g., your personal GitHub token under Settings)
- **Third-party confidential data** you are not authorized to share

5.2 What You CAN Use

- Internal engineering and delivery artifacts (requirements, design notes, architecture diagrams, code, configuration, test scenarios)
- Non-sensitive internal documentation and standards

5.3 Platform Safeguards

- **Internal-only access** via SSO; not exposed externally
- **Team-scoped isolation** — your data is not shared across teams
- Processing through **approved enterprise LLM Gateway** channels
- **No model training** on your inputs
- **Audit and usage monitoring** for appropriate use

You are responsible for ensuring that any document, prompt, or context you provide is appropriate for use with AI and free of Protected Data. If in doubt, leave it out.

6. Who Uses It?

Role	How They Use It
Developers and Engineers	Understand existing systems, generate implementation packages, and accelerate delivery work
Business Analysts and Product Owners	Generate structured requirements from source documents, discussions, and raw inputs
Delivery Leads and Orchestrators	Track initiatives, features, assignments, timelines, and cross-agent workflow visibility
QA Engineers	Generate BDD test scenarios and review delivery artifacts
Technical Leads and Platform Owners	Govern prompts, models, configurations, repositories, and team-level usage

Role	How They Use It
Support and Operations Teams	Produce summaries, draft responses, classifications, and day-to-day AI utilities

All users are **internal UHG / Optum workforce members** on onboarded delivery teams. The platform does not interact with members, patients, providers, or any external population.

7. Team Onboarding

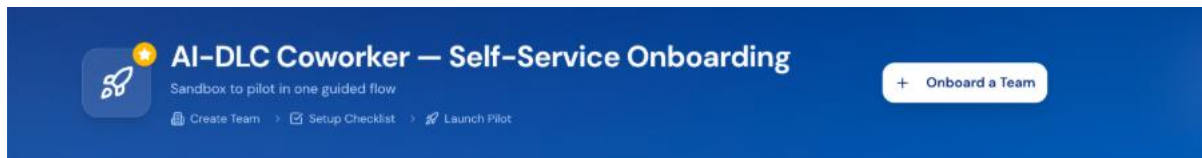
Getting Started

Onboarding is self-service. Submit your team onboarding request at the link below. The platform team will review your submission and notify you of the outcome.

 <https://aidlc-coworker.optum.com/onboarding>

Note: Screenshots reflect the interface available at the time of validation. Labels, navigation, and availability may change by release or configuration.

Onboarding request landing page





AI-DLC COWORKER
Onboard a Team

TEAM IDENTITY

Team Name * ⓘ

Avengers

3–100 chars. Letters, numbers, spaces, hyphens, &, . and @ allowed.

Workspace Owner (MSID) * ⓘ

vteja10

ASK ID * ⓘ

e.g. AIDE_OOxxx or UHGxxxx

Project or funding identifier associated with this team.

Workspace Type * ⓘ

Default

SECURE-GROUP APPROVERS ⓘ

Requester Employee ID ⓘ

001884815

Used as the secure-group owner.

Primary Approver Employee ID * ⓘ

001884815

Backup Approver Employee ID * ⓘ

e.g. 001887711

SECURE GROUPS THAT WILL BE CREATED ⓘ

AZU_COWORKER_AVENGERS_VIEWER

AZU_COWORKER_AVENGERS_EDITOR

PURPOSE & GOALS

Use Case * ⓘ

Leveraging AI-DLC Coworker to accelerate end-to-end delivery by automating requirements generation, architecture design, test scenarios, and operational runbooks. This enables faster onboarding, improved consistency, and reduced time-to-market across the full software development lifecycle.

Pilot Intent (optional) ⓘ

When submitting your request, you will be asked to provide:

Field	Description
Team Name	The name of your team workspace
Workspace Owner	The MSID of the team owner
ASK ID	Your project or funding identifier for cost attribution
Workspace Type	Select Default for general software teams, or ServiceNow for ServiceNow delivery teams
Secure Group Approvers	Enter the necessary details for the secure group approvers

AI-DLC Coworker | Internal Use Only | Version 1.0 | Updated 16th September 2026

Field	Description
Use Case	A brief description of what your team plans to use the platform for.
Pilot Intent	Optional description of what the team expects to achieve or validate during the pilot

Workspace Type matters. This selection determines which coding experience is available to your team inside CodeNow. See Section 18 for details.

Responsible AI note: Your stated use case is reviewed against approved use. Onboarding implies your team agrees to operate within the Responsible Use guidance in Section 1.

Onboarding Review

Once submitted, your request enters a review queue managed by the platform team. You will be notified when your request is approved or rejected. If rejected, a reason will be provided.

After Approval

Once approved, you will gain access to the onboarding dashboard. This dashboard tracks your team's setup progress and the steps required before your team can launch a pilot.

[Onboarding dashboard with setup progress](#)

Team Amavas
Approved

Setup progress: 1 of 4 required items ready

Secure Access Groups

As team owner, create & manage these groups — share them with your members so they can request the appropriate access level.

EDITOR

Full access — configure agents, settings & APIs

SVC_NOW_AGENT_AMAVAS_EDITOR

VIEWER

Read-only — view dashboards & agents

SVC_NOW_AGENT_AMAVAS_VIEWER

Setup Checklist

Refresh

Required items

1 / 4 ready

Workspace & Access
REQUIRED

Ready

Rally Setup
REQUIRED

Pending

Set up

Context & Graphify
REQUIRED

Pending

Set up

TCO Baseline
REQUIRED

Pending

Set up

Agent Harness (System Prompts & Custom Skills)

Optional

Set up

TCO Autonomous

Pending

Set up

GitHub

is connected individually per user under [Settings → Git Integration](#) once your pilot starts.

Complete required items first: **rally_setup, context_graphify, tco_baseline**

Start Pilot

Setup Progress Checklist

The **Start Pilot** action is gated. Your team cannot launch until all mandatory items are complete.

Setup Item	Required	Description
Workspace and Access	Yes	Secure groups created and team access confirmed
Rally Setup	Yes	Rally configuration connected to the team workspace
Context and Graphify Setup	Yes	Application context built and available for agents
TCO Baseline Setup	Yes	Traditional SDLC cost baseline recorded for comparison
Agent Harness	Optional	Custom system prompts and skills configured
TCO Autonomous Setup	Optional	AI-DLC velocity and cost capture configured

Complete all four required items before expecting full agent functionality. The dashboard will show warnings for any incomplete steps.

Next Steps for the Team Owner

As the team owner, you are required to create **two secure groups** as shown on the onboarding dashboard. Group names are automatically generated from your team's name using the following pattern:

AZU_COWORKER_<TEAMNAME>_VIEWER

AZU_COWORKER_<TEAMNAME>_EDITOR

Example — if your team's name is **Amavas**:

Group	Purpose
AZU_COWORKER_AMAVAS_VIEWER	Viewer access for team members
AZU_COWORKER_AMAVAS_EDITOR	Editor access for team admins and leads

8. Secure Group Ownership

After onboarding, a designated team owner is responsible for managing the secure groups for their team. This includes overseeing group membership and guiding team members on which group to request.

Access governance is a Responsible AI control. Only authorized internal users should be granted access. Owners must review membership periodically and remove access for members who no longer need it.

Recommended Ownership Model

Step	Action
1	The team owner identifies one or more secure group owners and creates the groups
2	Team members submit access requests to the appropriate group based on their role
3	Group owners approve or reject membership requests in line with team policy
4	Once group membership is active in SSO, users can sign in to the platform

How to Create a Secure Group

Follow the standard secure group creation process at <https://secure.uhc.com/>

33. Go to **Access Support > Self-Service Group Creation**

34. Configure the group with the following settings:

Field	Value
Platform	Windows
Group Type	Global
Domain	MS

Field	Value
Access Type	Non-Privileged

Secure group creation configuration

9. Access Roles

Each onboarded team has two access roles: **Viewer** and **Editor**.

Role	Group Example	What It Means
Viewer	AZU_COWORKER_AMAVAS_VIEWER	Access the team workspace, use agents, view team-level settings, and work with permitted team resources
Editor	AZU_COWORKER_AMAVAS_EDITOR	Everything a Viewer can do, plus the ability to manage team-level configurations and agent settings such as system prompts, context configurations, Rally settings, Git repositories, and Orchestrator setup

Editor membership automatically includes Viewer-level access. Users in the Editor group do not need separate Viewer group membership.

Practical Recommendation

Use Viewer access for most day-to-day team members. Use Editor access for team admins, technical leads, and platform owners who manage shared credentials, prompts, skills, Rally settings, Git repositories, model settings, and Orchestrator configuration.

Responsible AI note: Editors influence agent behavior for the **entire team** through prompts and skills. Editor access should be limited to trusted, accountable individuals.

10. Access Levels within the App

The app filters team visibility based on the secure groups in the user's active SSO session. A user can only see teams where they hold Viewer or Editor group membership.

Viewer Access

Viewers can:

- Sign in and access the team workspace
- See the team in the Team Selector
- Use all agents for their team
- View team details, prompt configuration, and skill configuration
- View connected system and repository configuration metadata
- View Orchestrator initiatives, sticky teams, features, and timelines
- View notifications
- Create personal connected-system configurations for their own use where enabled
- View shared configurations created by Editors

Viewers cannot manage shared team-level settings.

Access note: Beta, administrative, leadership, or restricted capabilities may be controlled separately. Availability can vary by workspace type, secure-group membership, pilot configuration, and feature-specific authorization.

Editor Access

Editors can do everything Viewers can do, plus:

- Create, update, and delete team-level connected-system configurations
- Mark configurations as shared or team-wide
- Create, update, delete, test, and set default Rally configurations
- Update, reset, and roll back prompt library prompts
- Manage Git repository configurations, model settings, and agent skills
- Reassign Orchestrator features
- Create and manage initiatives and sticky teams

11. Signing In

35. Open <https://aidlc-coworker.optum.com/>

36. Sign in with SSO
37. If authorized, the app opens to your AI-DLC Coworker workspace
38. If you see an unauthorized page, confirm your secure group membership has been approved and synced through SSO
39. Use the **Team Selector** to choose your onboarded team

12. First-Time Team Setup

Before the full workflow is available, an Editor should complete the team's initial setup. This maps directly to the onboarding checklist on your dashboard.

Step	Action
1	Confirm team members are in the correct Viewer or Editor secure group
2	Configure Rally integration by providing your Rally Project ID and API key
3	Upload context documents including architecture diagrams, coding standards, and domain reference material
4	Run Graphify on your repository to generate a knowledge graph
5	Enter your TCO baseline to record traditional SDLC effort and cost estimates
6	Add Git repository configuration if generated code should be pushed to a repository
7	Review and customize CodeNow prompts if your team has specific coding standards

Once these steps are complete, your team is ready to launch the pilot and begin working through the delivery flow described in the sections below.

Before uploading anything: confirm context documents contain **no PHI, PII, or Protected Data** (see Section 5).

13. Step 1: Set Up Your Context

Before running any agents, your team needs to give them the right background to work from. This is done through context setup.

What Is Context?

Context is the background knowledge agents use to generate relevant, accurate output. Without it, agents have no understanding of your product, codebase, or domain and will produce generic responses.

Context comes from two sources:

Uploaded Documents

Upload any documents that describe your product, system, or domain. This can include:

- Architecture diagrams
- Coding standards and guidelines

- Domain reference material
- Meeting transcripts and discussion notes
- Spreadsheets and data models
- PDFs and images

Editors manage shared context files. Active context files are injected into agent sessions at runtime.

Graphify: Knowledge Graph from Your Codebase

Graphify is a utility that generates a structured knowledge graph from your code repository. It maps files, services, components, APIs, dependencies, and relationships across your backend and frontend repositories, giving agents an application-level understanding of your codebase.

This helps agents answer questions like:

- What components are affected by this change?
- What is the blast radius of modifying this table or service?
- What patterns already exist in the codebase for this type of work?

Graphify is optional but strongly recommended for teams that want agents to reason about their codebase rather than just their documents.

How to Set Up Context

40. Go to **Settings > Context Configuration**
41. Upload relevant documents for your team
42. Mark documents as active so they are included in agent sessions
43. To set up Graphify, follow the instructions in the Graphify setup section of your onboarding dashboard
44. Once Graphify runs, the generated knowledge graph is committed and made available to agents automatically

 **Data rule:** Do **not** upload documents containing PHI, PII, or any sensitive regulated data. You are responsible for ensuring that context documents uploaded to the platform are appropriate for use with AI agents. **Context quality directly affects output quality — and bad or inappropriate context creates risk.**

14. Step 2: Explore with Playground Agent

Once your context is set up, the best starting point for any new team member or new feature area is the **Playground Agent**.

What It Does

Playground Agent is a conversational explorer for your team's connected systems, configurations, repositories, and uploaded context. It lets users ask natural-language questions about existing structures, metadata, scripts, workflows, rules, and implementation details.

Use Playground when you are new to a product area, want to understand what already exists before writing requirements or code, or need quick answers from your team's uploaded context and knowledge graph.

Responsible AI note: Playground is **read-only** and **advisory**. Its answers may be incomplete or incorrect. **Verify any answer** against authoritative sources before acting on it. It does not trigger any automated action.

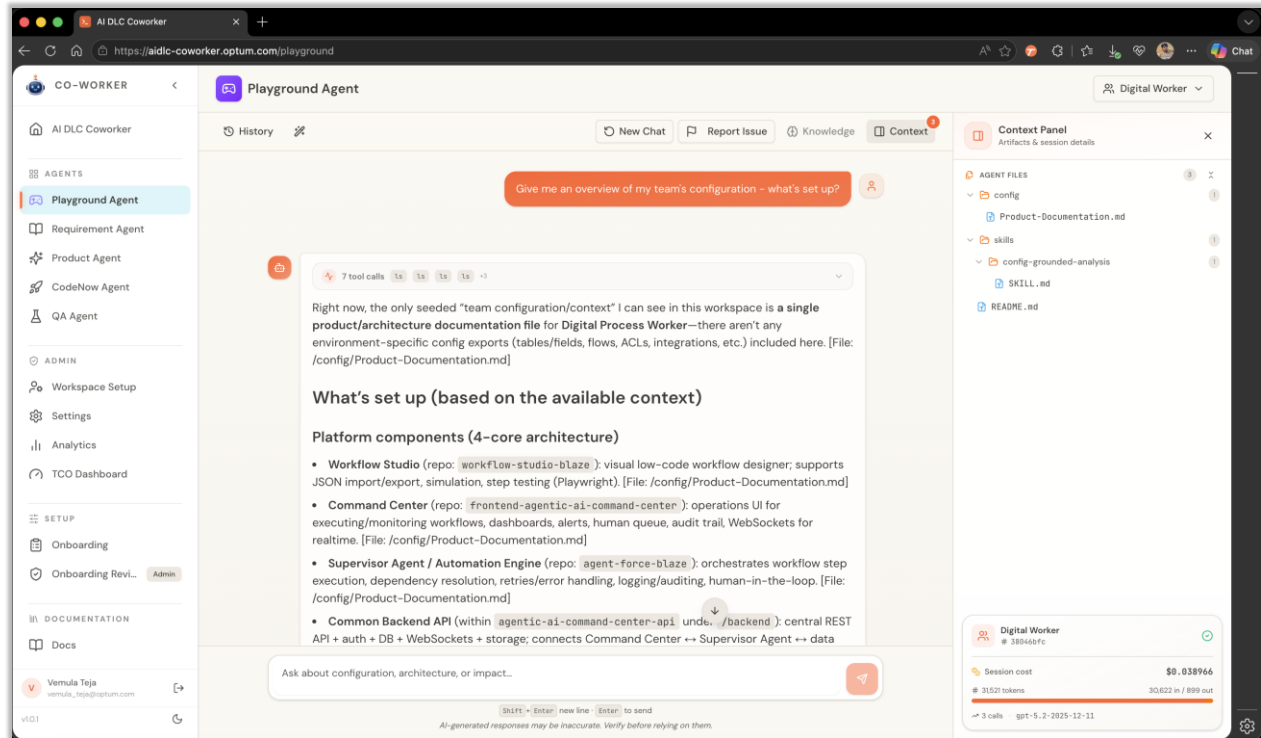
How to Use It

45. Open **Playground**
46. Select your team
47. Start a new chat or continue an existing conversation
48. Ask a question about your connected system, repository, or uploaded context
49. Review the response and tool activity
50. Use the answer as context for requirements, development, or onboarding discussions

Example Prompts

- "What workflows already exist for this process?"
- "Where is this field or object referenced in the codebase?"
- "Explain the scripts and automation related to this feature."
- "Summarize the architecture of this component based on the uploaded documents."
- "What controls or rules affect access to this component?"

[Playground Agent conversation view](#)



15. Step 3: Set Up Your Workspace

With context in place and a basic understanding of the system, the next step is to organize your delivery work inside **Workspace Setup**.

What Is Workspace Setup?

Workspace Setup is the workflow control plane for AI-DLC COWORKER. It organizes delivery work into initiatives and tracks everything that happens to a feature across all agents. Every agent action, from requirement generation to PR merge, is recorded in the feature timeline here.

Traceability is a Responsible AI control. The feature timeline preserves a record of AI outputs and the human decisions taken on them — supporting accountability and auditability.

Key Concepts

Initiative

An initiative is the top-level container for a body of work. Think of it as a project or epic. All features, assignments, and timelines live under an initiative.

If your team works across multiple products or capabilities — for example a recharge flow and a ticketing flow — create separate initiatives for each. They share the same team workspace but maintain independent features and timelines.

Pod

A Pod is a small group of team members assigned to an initiative — typically 3 to 4 people. The Pod is responsible for reviewing and acting on agent outputs for features under that initiative.

Round-Robin Assignment

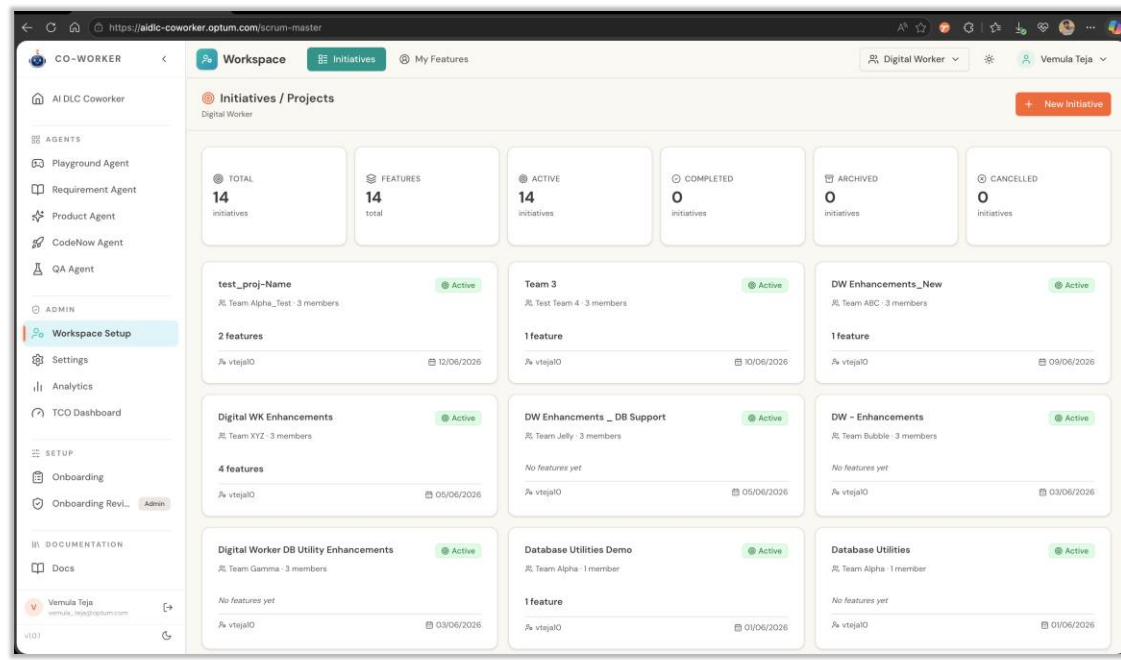
When a new feature is created under an initiative, it is automatically assigned to the Pod member using round-robin assignment. Assigned members are notified by email. This removes the need for manual triage and ensures even distribution of work across the team.

How To Set Up Your Workspace

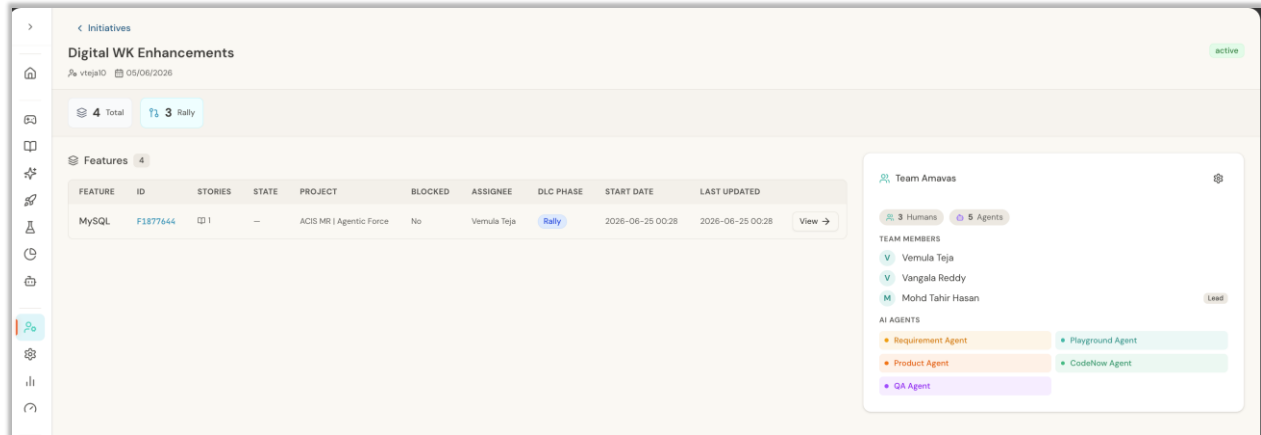
51. Open **Workspace Setup**
52. Select your team
53. Click **Create Initiative** and give it a name and description
54. Create a **Pod** under the initiative
55. Add 3 to 4 team members to the Pod. Members can be added by MSID or invited by email if not yet registered
56. Assign roles within the Pod: Lead or Member
57. Configure notification preferences for each member

Once your initiative and Pod are set up, you are ready to start creating features.

Workspace Setup / Initiative view



Pod assignment view



16. Step 4: Generate Requirements

With your initiative and Pod in place, the next step is to generate requirements for the features you want to build. This is done using the **Requirement Agent**.

What It Does

Requirement Agent is the notebook-based requirements assistant. It helps teams turn scattered source material into structured, versioned, and approved requirement documents.

It supports:

- Uploading source documents including Word files, PDFs, PowerPoint presentations, meeting transcripts, spreadsheets, and images
- Asking clarifying questions and refining requirements through conversation
- Generating structured requirement documents
- Comparing requirement versions with section-level diffs
- Approving or rejecting requirement candidates before downstream use
- Pushing approved requirements directly to Rally as features

Use it when business requirements are unclear, scattered across documents, or need to become a reviewable technical requirement before development begins.

Human-in-the-loop: Generated requirements are **candidates only**. Nothing is pushed to Rally until a human reviews the section-level diff and explicitly approves.

How to Use It

58. Open **Requirement Agent**

59. Select your team

60. Create a notebook. Link it to your initiative in Workspace Setup

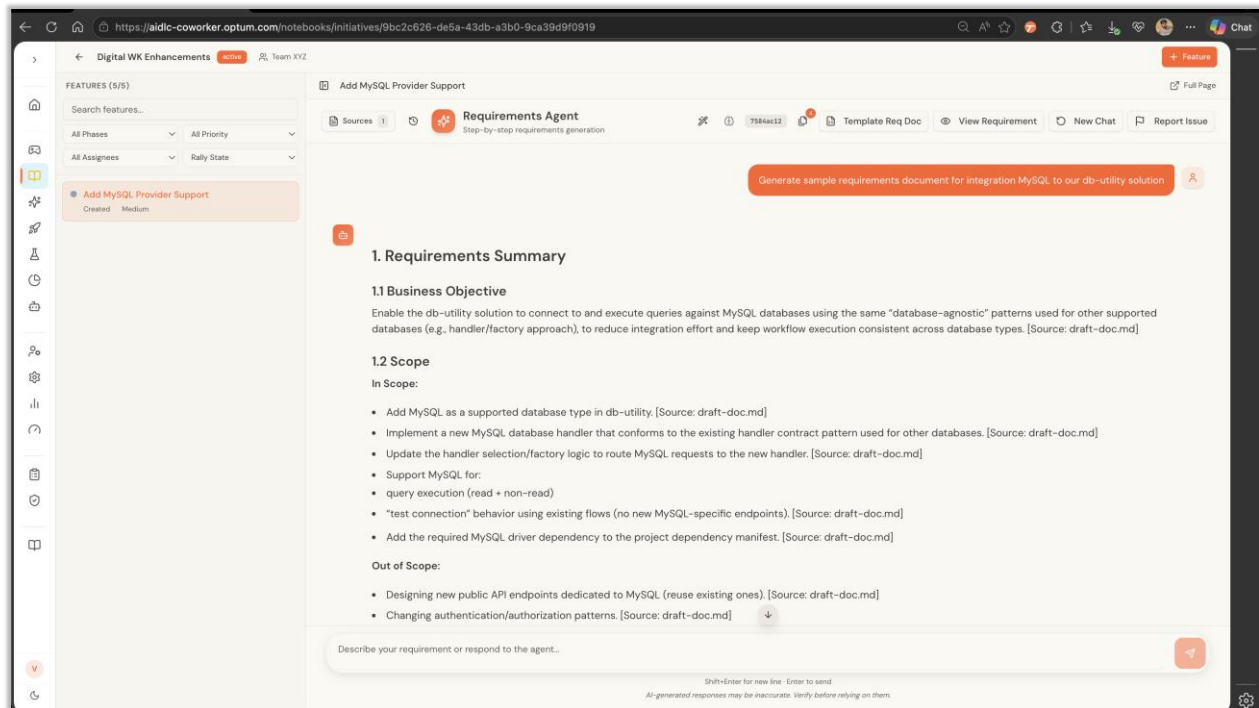
61. Upload source documents or paste text sources relevant to the feature
62. Ask the agent to analyze the sources and generate a draft requirement
63. Ask follow-up questions to refine or clarify
64. Generate the requirement document
65. Review the pending candidate and section-level diff against any prior version
66. **Approve or reject** the requirement
67. Use the Rally push flow to create the feature in Rally under your project

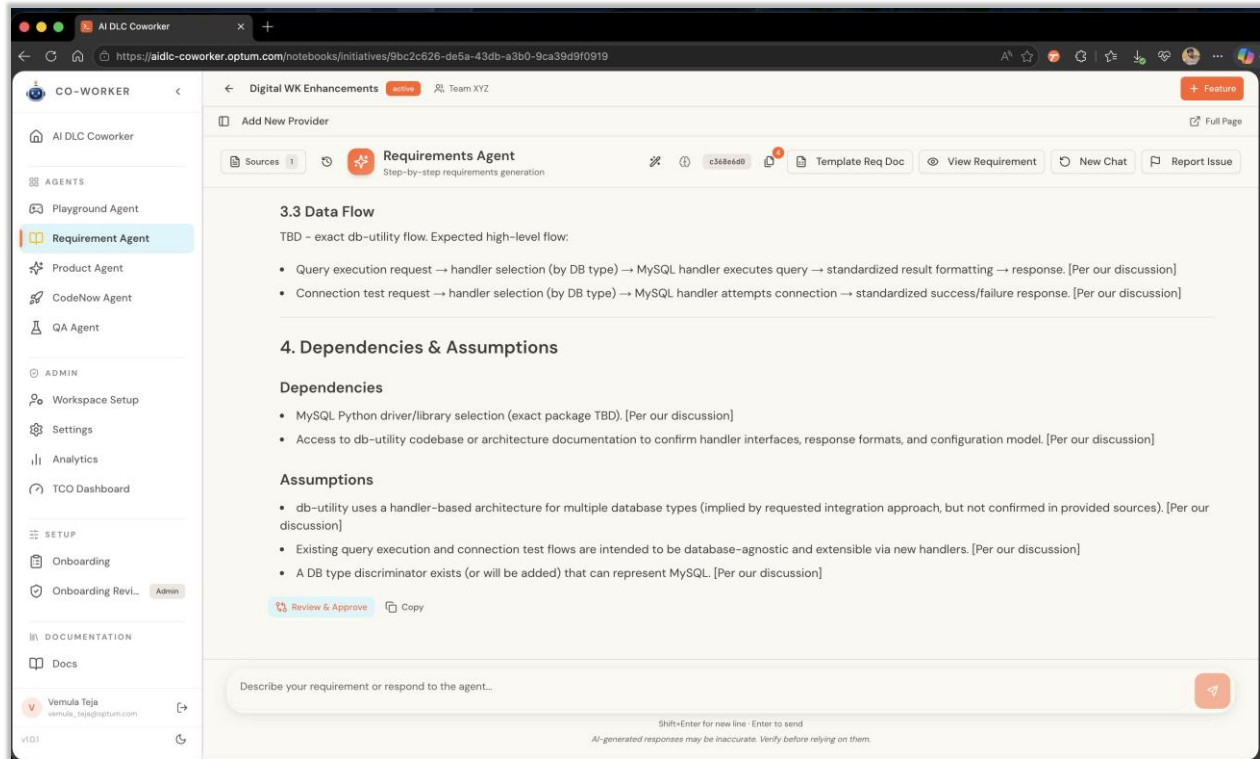
Once the requirement is approved and pushed to Rally, it becomes the traceability anchor for all downstream work. The feature is automatically assigned to the next Pod member via round-robin.

Example Prompts

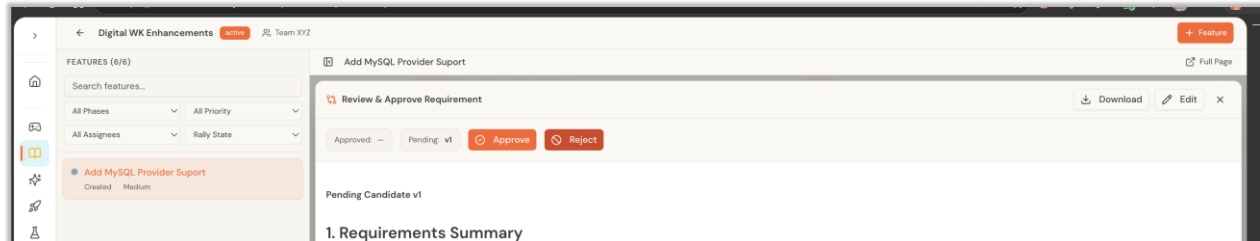
- "Summarize the business requirement from all uploaded documents."
- "Generate a requirements document based on the uploaded context."
- "What open questions should we resolve before development starts?"
- "Compare this version against the previously approved requirement."
- "Are there any gaps or risks in the current requirement?"

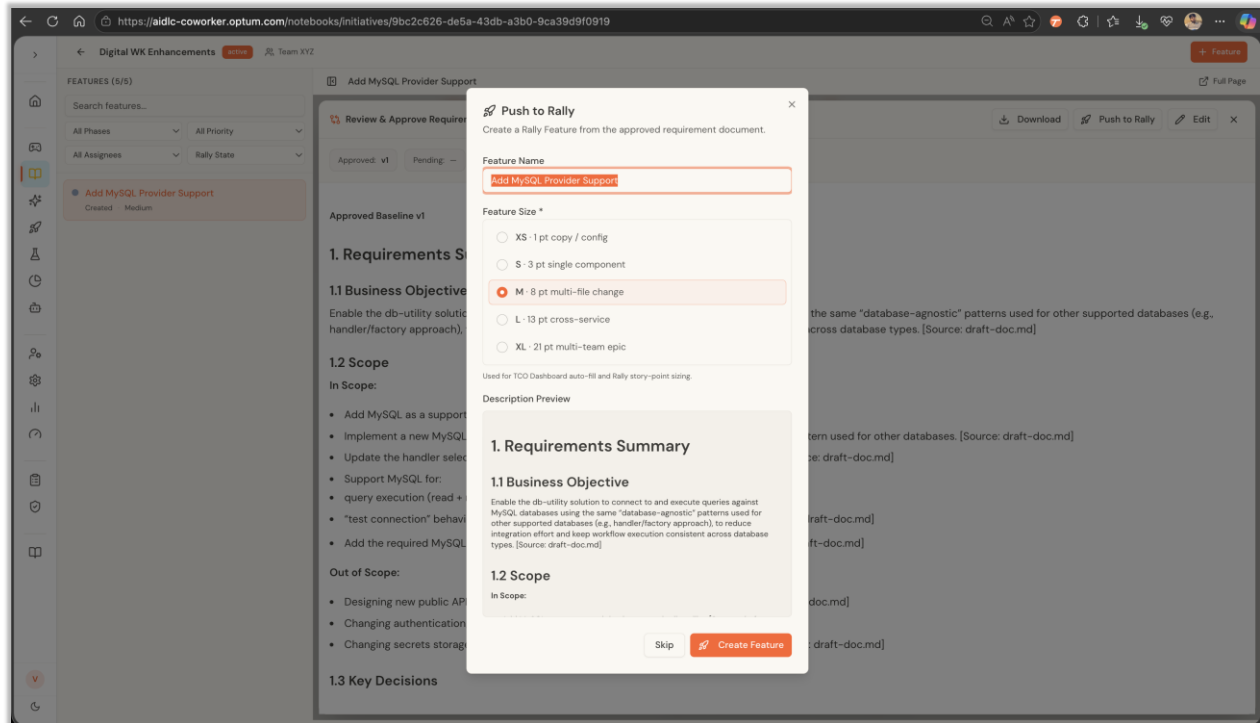
Requirement Agent notebook view





Requirement candidate diff / approve-reject view





17. Step 5: Break Down Features with Product Agent

Once a feature exists in Rally, the **Product Agent** can break it down into user stories and tasks ready for development.

What It Does

Product Agent processes Rally Features and produces reviewable user stories and tasks. It is designed for **human-in-the-loop** product breakdown. The agent drafts the breakdown and a human reviews and approves before anything is written to Rally.

The agent loads features assigned to your team from Rally, generates stories and tasks based on feature complexity and any additional context you provide, and holds everything in a review session until you submit your decisions.

How to Use It

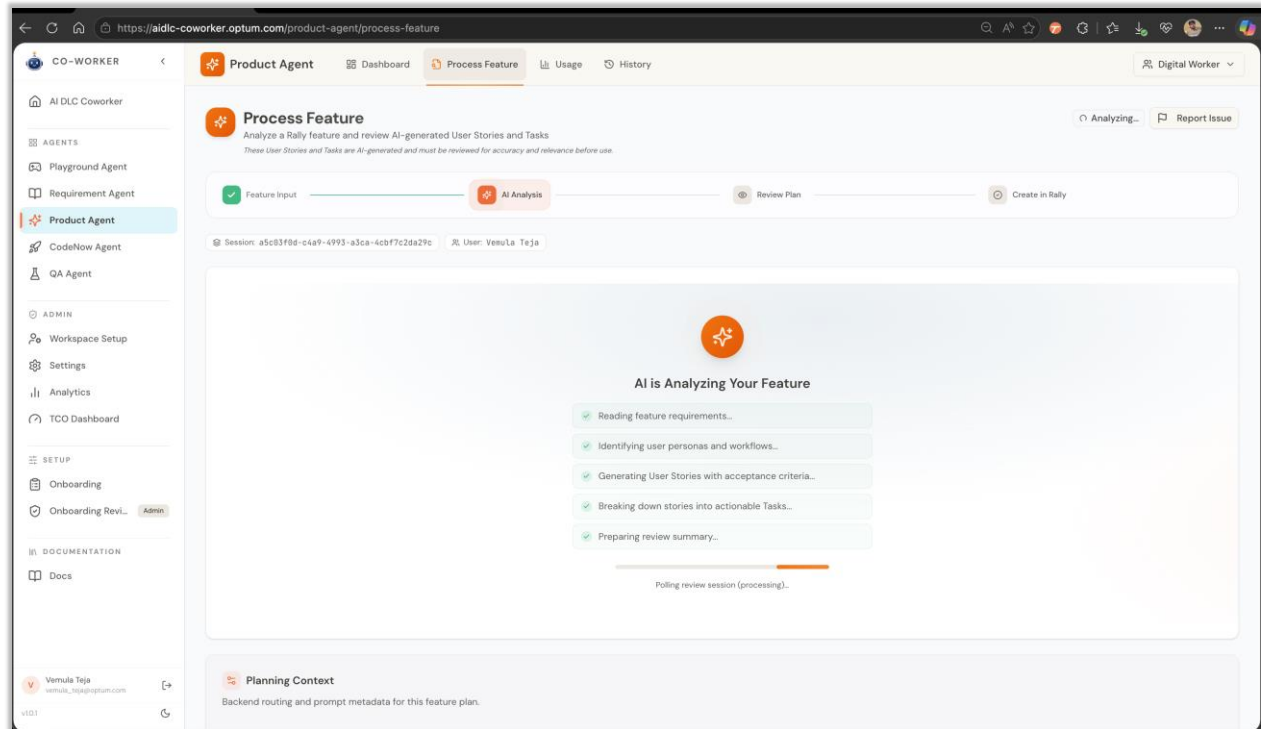
68. Open **Product Agent**
69. Select your team
70. Choose an available Rally Feature from the list or enter a Feature ID manually
71. Enter the product name and any additional context that should guide the breakdown
72. Start processing
73. Wait for the review session to become ready
74. Review generated stories and tasks

75. **Approve, reject, or edit each item** individually
76. Use Approve All or Reject All where appropriate
77. Submit your decisions. Approved items are created in Rally under the relevant feature
78. Confirm that the Workspace Setup feature timeline records the Product Agent result

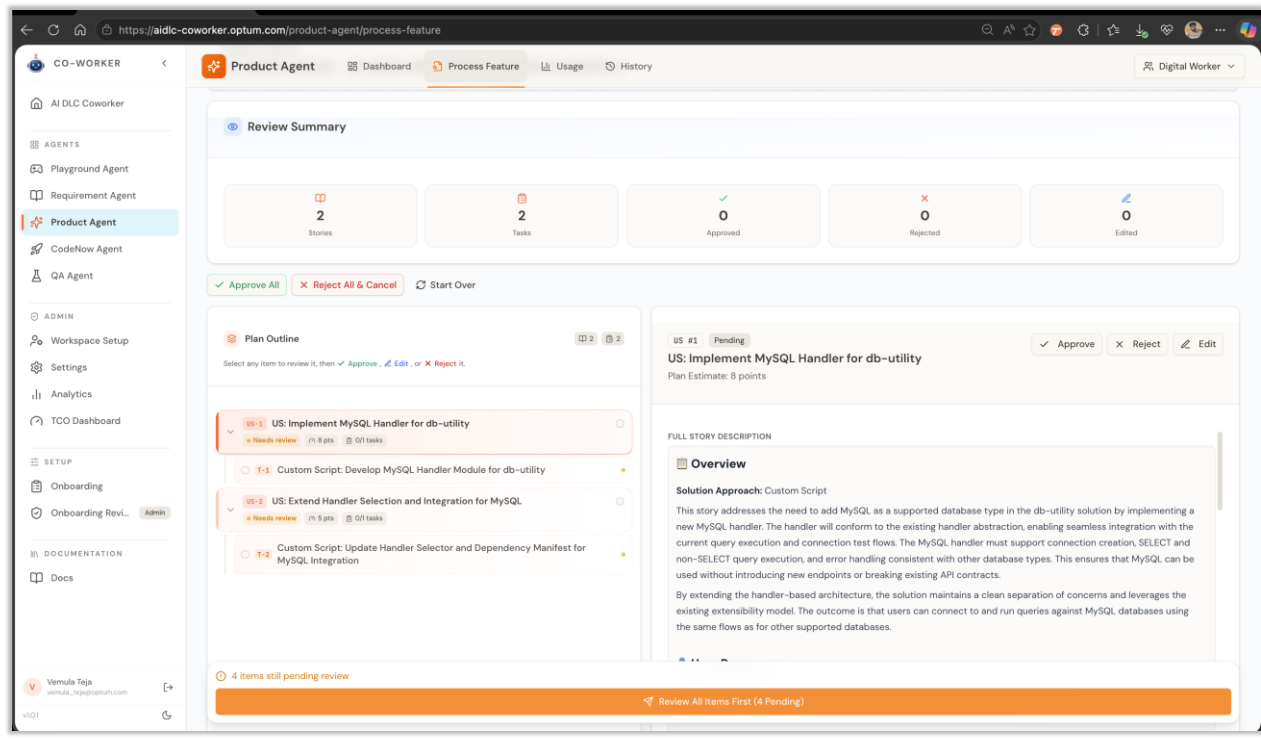
The quality of story and task generation improves with richer feature descriptions and uploaded context. If generated stories feel too broad or too granular, adjust the feature description or add more context before reprocessing.

Product Agent feature selection

The screenshot displays the 'Product Agent' interface within the 'AI-DLC Coworker' application. The left sidebar contains navigation links for 'AGENTS' (Playground Agent, Requirement Agent, Product Agent, CodeNow Agent, QA Agent), 'ADMIN' (Workspace Setup, Settings, Analytics, TCO Dashboard), 'SETUP' (Onboarding, Onboarding Rev...), and 'DOCUMENTATION' (Docs). The main content area is titled 'Process Feature' and includes a progress bar with steps: Feature Input, AI Analysis, Review Plan, and Create in Rally. Below this, the 'Feature Details' section prompts the user to 'Pick an Assigned Feature' and provides a search bar for 'Search Feature ID'. A feature with ID 'F1877723' is listed with the description '1. Requirements Summary 11 Business Objective Enable the existing db-utility solu...' and is marked as 'Selected'. To the right, the 'Team' configuration section includes fields for 'Feature ID' (F1877723), 'Number of User Stories (optional)' (2), 'Product Name (optional)' (ACIS MR | Agentic Force), and 'Additional User Context (optional)' (Generate 1 task for each userstory). An 'Analyze Feature' button is located at the bottom right of the team configuration area.



Story / task review session with per-item controls



Plan Outline

2

2

Select any item to review it, then ✓ Approve, ✎ Edit, or ✗ Reject it.

US-1

US: Implement MySQL Handler for db-utility

8 pts

1/1 tasks

✓

✓ T-1

Custom Script: Develop MySQL Handler Module for db-utility

US-2

US: Extend Handler Selection and Integration for MySQL

5 pts

1/1 tasks

✗

✗ T-2

Custom Script: Update Handler Selector and Dependency Manifest for MySQL Integration

US #1

✓ Approved

✓ Approve

✗ Reject

✎ Edit

US: Implement MySQL Handler for db-utility
 Plan Estimate: 8 points

FULL STORY DESCRIPTION

Overview

Solution Approach: Custom Script
 This story addresses the need to add MySQL as a supported database type in the db-utility solution by implementing a new MySQL handler. The handler will conform to the existing handler abstraction, enabling seamless integration with the current query execution and connection test flows. The MySQL handler must support connection creation, SELECT and non-SELECT query execution, and error handling consistent with other database types. This ensures that MySQL can be used without introducing new endpoints or breaking existing API contracts.

 By extending the handler-based architecture, the solution maintains a clean separation of concerns and leverages the existing extensibility model. The outcome is that users can connect to and run queries against MySQL databases using the same flows as for other supported databases.

User Persona

Submit Review (2 Approved, 2 Rejected, 0 Edited)

Plan Outline

2

2

Select any item to review it, then ✓ Approve, ✎ Edit, or ✗ Reject it.

US-1

US: Implement MySQL Handler for db-utility

8 pts

1/1 tasks

✓

✓ T-1

Custom Script: Develop MySQL Handler Module for db-utility

US-2

US: Extend Handler Selection and Integration for MySQL

5 pts

1/1 tasks

✗

✗ T-2

Custom Script: Update Handler Selector and Dependency Manifest for MySQL Integration

US #2

✗ Rejected

✓ Approve

✗ Reject

✎ Edit

US: Extend Handler Selection and Integration for MySQL
 Plan Estimate: 5 points

FULL STORY DESCRIPTION

Overview

Solution Approach: Custom Script
 This story covers the extension of the handler selection mechanism (factory/registry) to support MySQL as a database type. It ensures that when a request specifies MySQL, the correct handler is returned and integrated into the existing query execution and connection test flows. The solution must not introduce MySQL-specific endpoints or alter the API layer, maintaining database-agnostic flows.

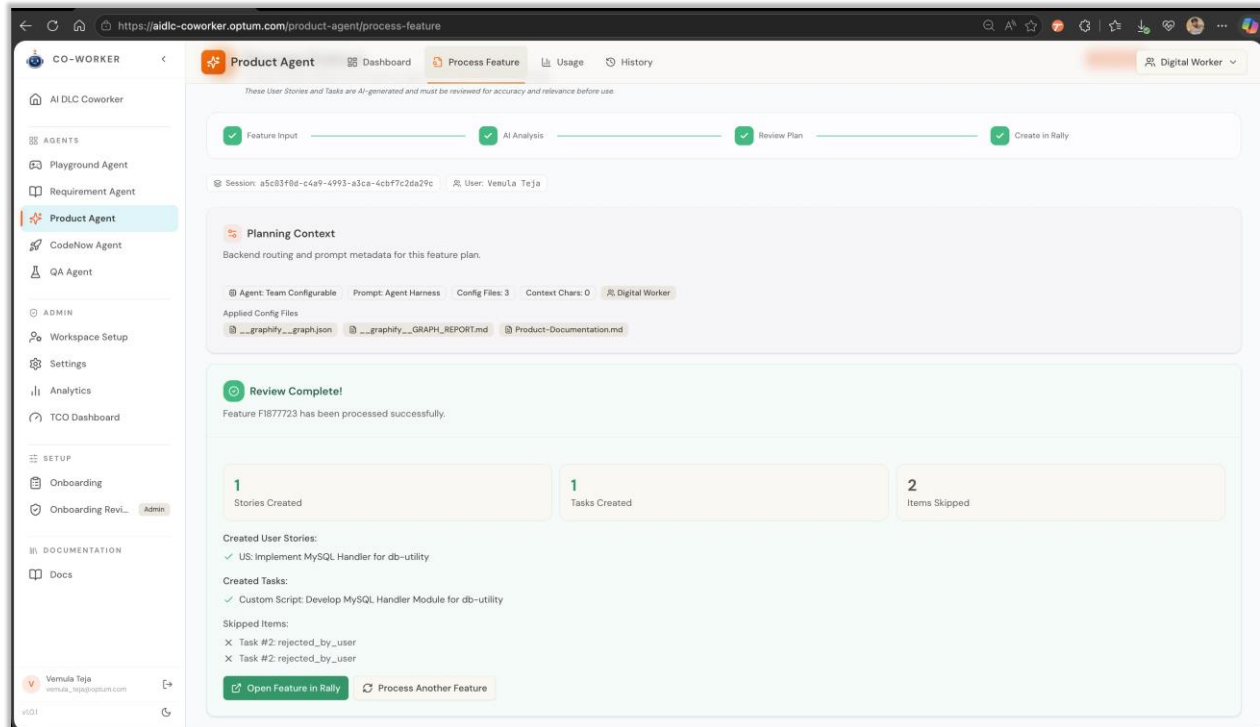
 By updating the handler selector and ensuring compatibility with the connection test and query execution flows, the system will support MySQL in a seamless, maintainable way. This approach leverages the existing architecture and ensures that MySQL is treated as a first-class citizen alongside other supported databases.

User Persona

As a platform engineer, I want the handler selector to support MySQL, so that MySQL can be used in all standard db-

Submit Review (2 Approved, 2 Rejected, 0 Edited)

Submission confirmation / Rally creation



18. Step 6: Implement with CodeNow Agent

With stories and tasks in Rally, developers can begin implementation using the **CodeNow Agent**.

What It Does

CodeNow Agent is the primary coding experience in AI-DLC Coworker. It analyzes requests, inspects context, generates implementation artifacts, and packages output for human review.

The coding experience available to your team depends on the workspace type selected during onboarding.

CodeNow does not directly deploy to downstream systems. All output is reviewable, and all generated code must pass your team's standard **code review, security scanning, validation, and release** processes before use.

Default Teams: Delegations to GitHub Copilot cloud agent

Teams onboarded with the **Default** workspace type use **Delegations** as their primary coding path inside CodeNow.

Delegations lets you hand off a scoped coding task to GitHub Copilot cloud agent for a connected repository where the user and required agent integration have the necessary

permissions. The delegated work returns as reviewable repository changes or a pull request for human approval.

The delegation workflow is designed to support repository-based work across multiple languages, frameworks, and runtimes. Results depend on repository structure, available context, agent compatibility, permissions, and task complexity.

How Delegations work:

79. Select a Rally task or write a custom prompt describing the work
80. Choose the target repository and base branch
81. Create the delegation. The platform raises a GitHub Issue and assigns it to GitHub Copilot cloud agent with the task description, target repository, base branch, and any custom instructions
82. GitHub Copilot cloud agent creates a branch, implements the change, and opens a pull request
83. The platform tracks the delegation lifecycle from issue creation through to PR merge
84. **Review the pull request and merge after approval** following your team's normal engineering process

Known consideration: Repository activity created by the delegated coding agent may be attributed to the agent in GitHub history. Teams should verify attribution and audit requirements. If PAT-based operations fail, capture the affected operation and report it to the Platform Team.

Delegation lifecycle statuses:

Status	Meaning
Pending	Delegation created, issue not yet raised
Queued	Issue raised, agent not yet started
In Progress	Agent is actively working
PR Opened	Pull request created
Ready for Review	PR is ready for human review
Merged	PR merged, delegation complete
Closed Without Merge	PR closed without merging
Cancelled	Delegation cancelled by user
Failed / Timed Out	Agent did not complete successfully

GitHub Token Setup

Each user must configure their own GitHub personal access token under **Settings > Git Integration** to enable Delegations. This is required for the platform to interact with your repositories on your behalf. GitHub integration becomes available after your pilot is started.

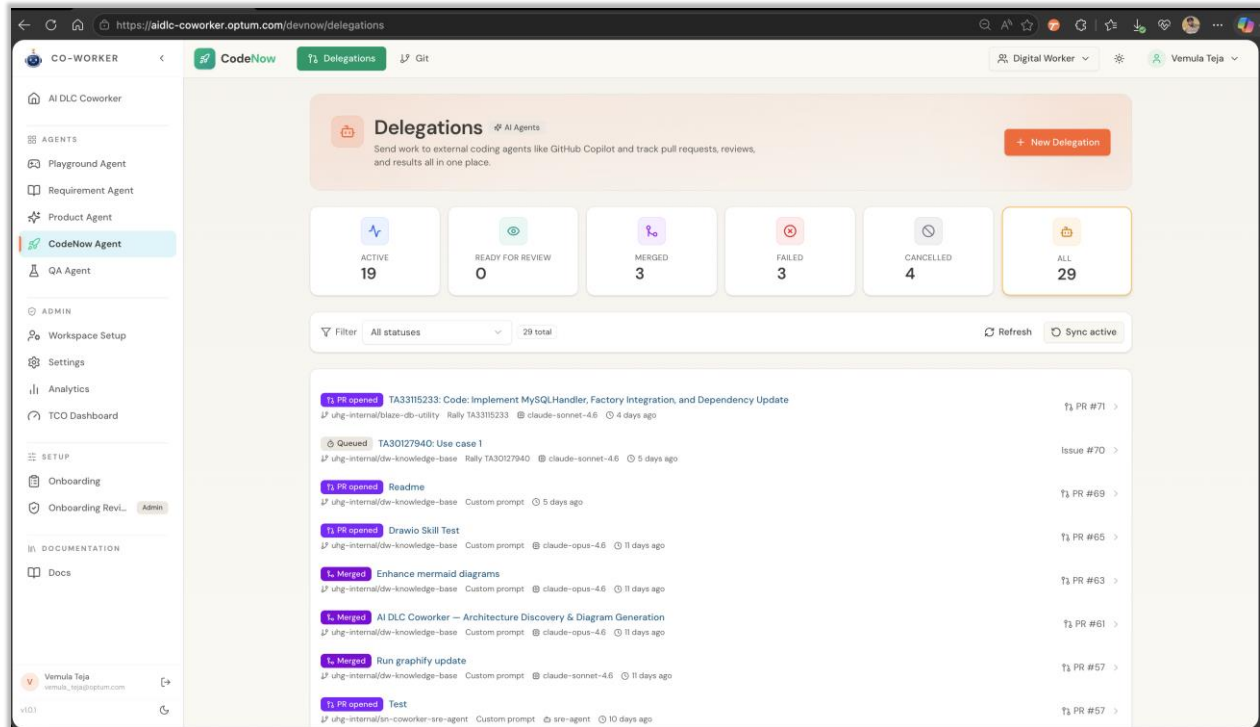
Security note: Treat your personal access token as a credential. Do not share it. Scope it to the minimum access required.

Delegation creation form

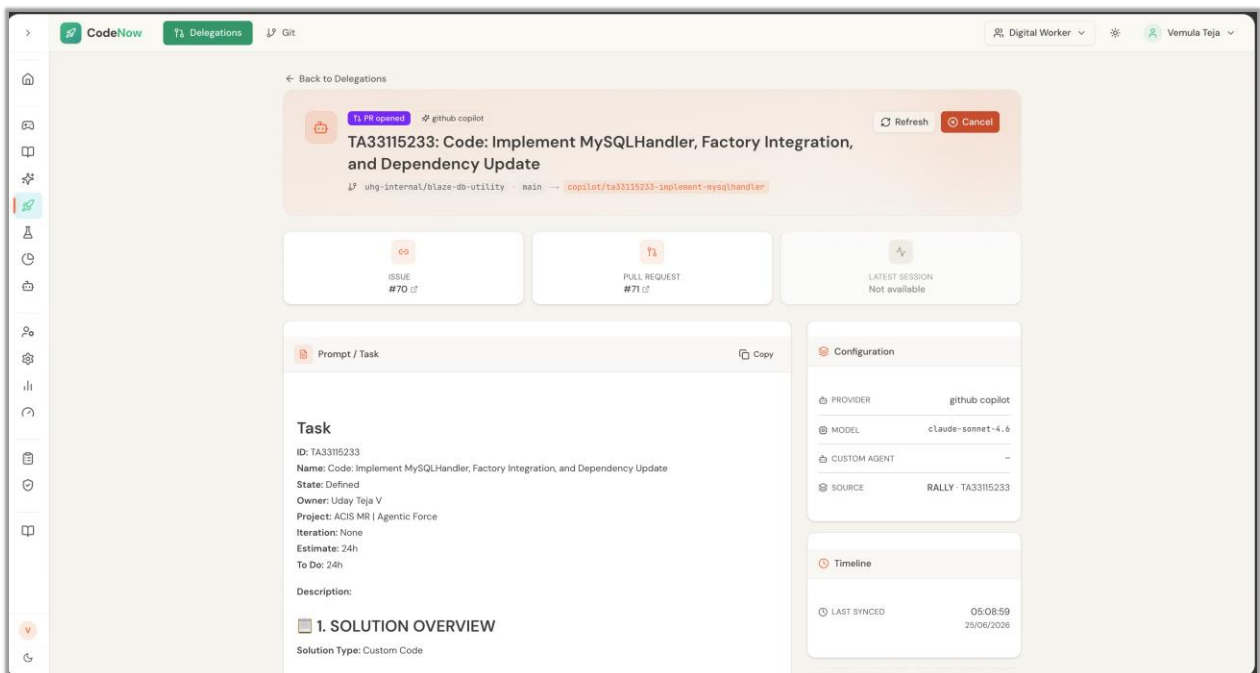
The screenshot shows the 'New Delegation' form in the AI-DLC Coworker interface. The form is titled 'New Delegation' and includes a subtitle 'Delegate work to a GitHub Copilot coding agent — from Rally tasks or a custom prompt.' Below the title, there are two tabs: 'Rally Tasks' (selected) and 'User Prompt'. The 'Rally Tasks' tab displays a list of tasks under the heading 'Select Tasks'. The tasks are listed in a table with columns for 'In progress' and 'Defined'. The first task is 'TA33164815 Defined' with a custom script 'Develop MySQL Handler Module for db-utility'. Below the task list, there are buttons for 'Import user story' and 'Import feature'. At the bottom of the form, there is a 'Preview & Create' button. The interface also shows a sidebar with navigation options like 'AGENTS', 'ADMIN', 'SETUP', and 'DOCUMENTATION'.

In progress	Defined
☒	☐
TA33164815 Defined Custom Script: Develop MySQL Handler Module for db-utility F1877723: Add New Provider US10734802: US: Implement MySQL Handler for db-utility Import user story Import feature	
☐	☐
TA33157211 Defined Custom Script: Implement MySQL Handler and integrate with Handler Selection F1877723: Add New Provider US10731279: US: Integrate MySQL Support into db-utility via Handler Architecture	
☐	☐
TA33116788 Defined Custom Script: Implement MySQLHandler and integrate with Handler Factory and Dependency Management F1877648: Test MySQL US10719135: US: Enable MySQL Database Support via Handler-Based Architecture	
☐	☐
TA30127948 Defined Use case 1	

Delegation lifecycle / status tracking



Resulting pull request view



ServiceNow Teams: CodeNow Studio

Teams onboarded with the **ServiceNow** workspace type have access to **CodeNow Studio**, a domain-specialized coding experience built for ServiceNow delivery.

CodeNow Studio uses an orchestrator and a set of specialist subagents to provide platform-aware reasoning across ServiceNow schema, scripting, configuration, and testing. Rather than treating ServiceNow as a generic codebase, CodeNow Studio understands table structures, field relationships, scripting patterns, and platform conventions.

Internal Specialist Agents:

Specialist	Role
Schema Analyst	Analyzes table schemas, field reuse opportunities, parent/child relationships, choice lists, and reference fields
Script Writer	Generates GlideScript including Business Rules, Client Scripts, and Script Includes, grounded in schema analysis
Impact Analyzer	Assesses downstream effects of proposed changes on existing configurations, integrations, and workflows
Code Reviewer	Reviews generated code against ServiceNow platform standards, security concerns, and performance patterns
Test Generator	Produces test scripts validating the implementation against the original requirement

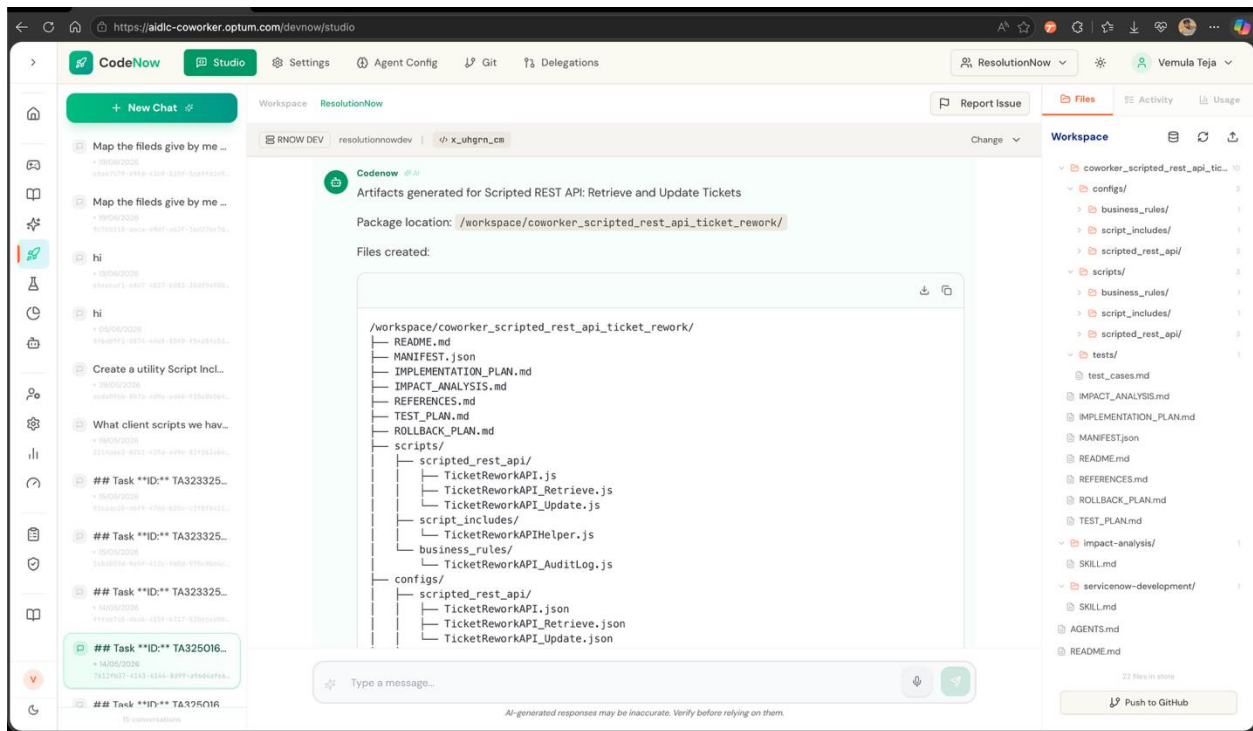
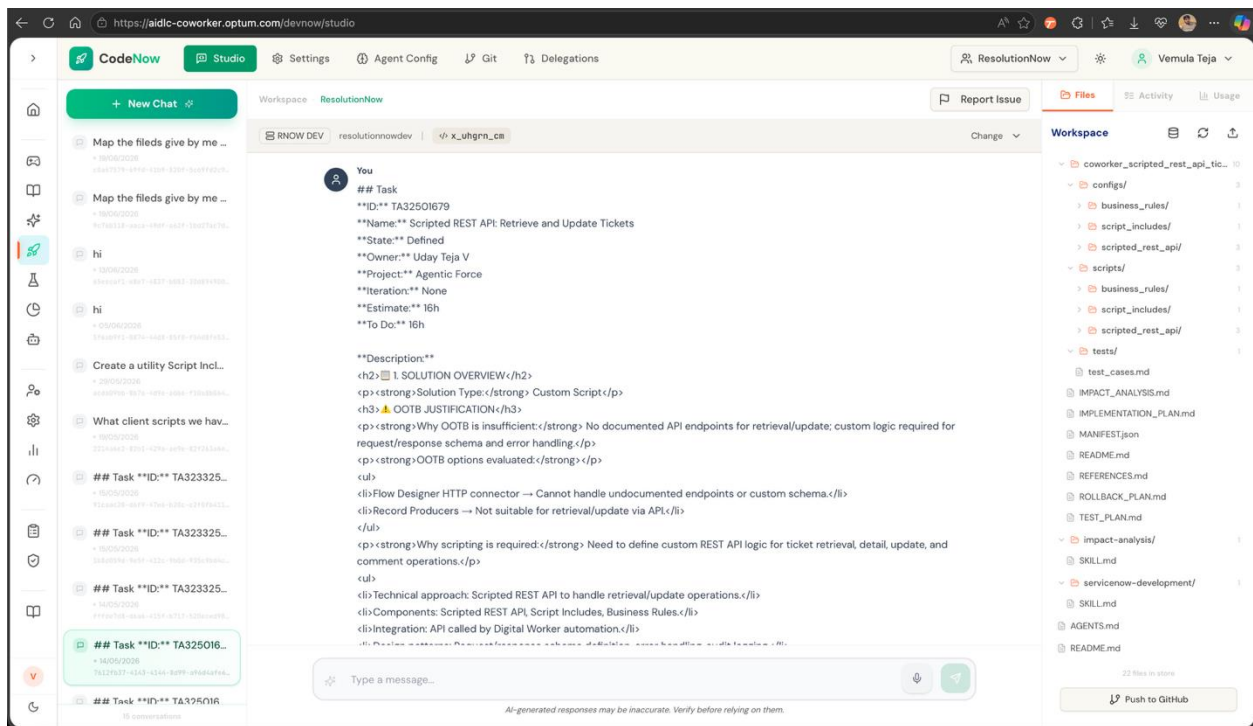
You do not need to call these specialists directly. The orchestrator delegates to them as needed and streams all activity back to you in real time. You can interrupt at any point to provide decisions or corrections.

Even with the built-in Code Reviewer specialist, a human engineer must still review and validate all generated code. The AI reviewer is an assist, not a replacement for human review.

How To Use CodeNow

1. Open **CodeNow**
2. Select your team
3. Select the relevant connected-system configuration or repository context
4. Start with a prompt, Rally task, or feature/story context
5. For ServiceNow teams: submit the request to CodeNow Studio for interactive, platform-aware implementation
6. For Default teams: use Delegations to dispatch the task to GitHub Copilot cloud agent
7. Watch the activity stream for model responses, tool calls, subagent activity, todos, and cost updates
8. Review the generated workspace files, branch changes, or pull request
9. Continue through your team's normal review, validation, and deployment process

CodeNow / CodeNow Studio activity stream



What CodeNow Can Generate

- Implementation files and code changes
- Configuration artifacts
- Supporting scripts and automation updates

- Impact analysis
- Test plans
- Rollback plans
- Deployment or release package manifests
- Repository-ready changes for delegated cloud-agent workflows

19. Step 7: Generate Tests With QA Agent

Once implementation is ready for review, the **QA Agent** can generate BDD test scenarios to validate the work against the original acceptance criteria.

What It Does

QA Agent (also shown as **Testing Agent** in the UI) creates reviewable BDD-style test cases for user stories. It supports human review, edits, regeneration, final approval, and automation tracking.

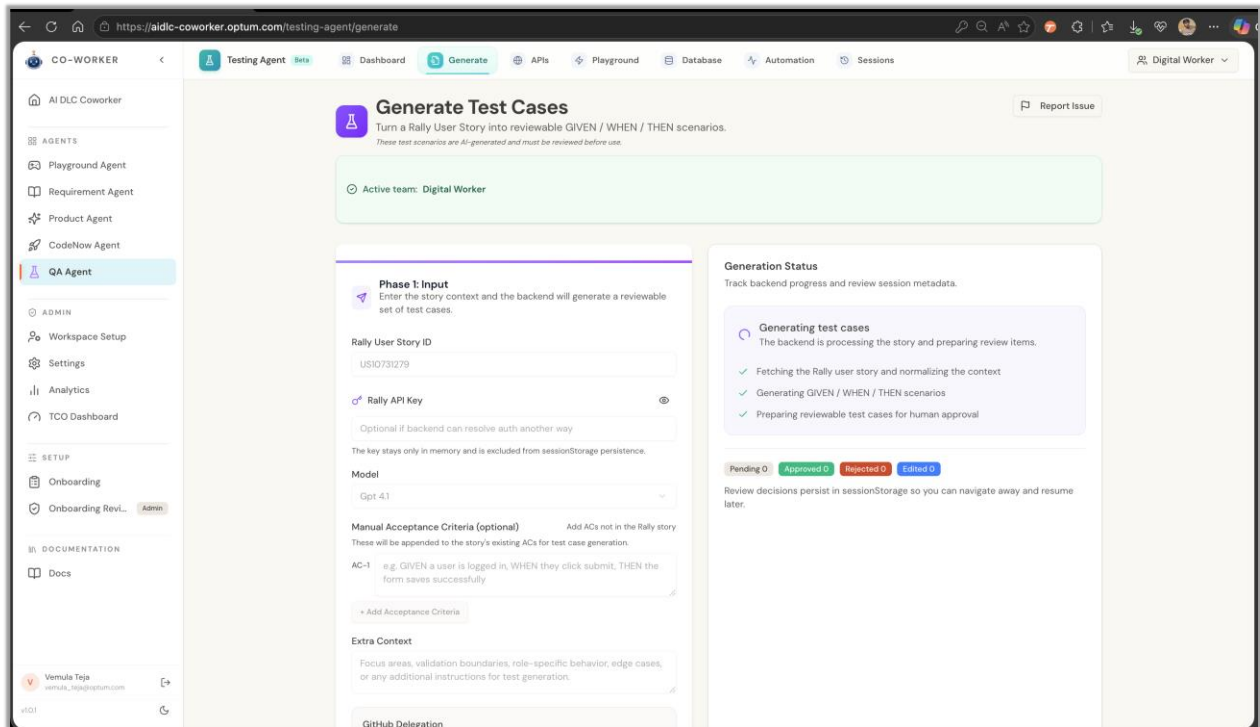
The QA Agent supports test case generation, human review, regeneration, automation delegation to GitHub Copilot cloud agent, API testing, and regression workflow execution. Automated test data generation is planned for future releases.

How to Use It

1. Open **QA Agent**
2. Select your team
3. Enter or select a story
4. Add product name, team name, number of test cases, and any optional context
5. Generate test cases
6. Review each scenario in Given, When, Then structure
7. **Approve, reject, or edit** test cases individually
8. Optionally regenerate approved scenarios for final review
9. Submit the final review
10. Use automation tracking views if your team has automation integration enabled

Human-in-the-loop: Generated test scenarios are drafts. A QA engineer must validate coverage and correctness before they are relied upon.

QA Agent BDD scenario review (Given/When/Then)



Test Types Generated

The Testing Agent generates structured BDD scenarios covering the following test categories for each user story:

- Functional and happy-path scenarios validating core business logic
- Negative and validation scenarios covering error handling and invalid inputs
- Edge and boundary scenarios testing limits and corner cases
- Integration and data-flow scenarios verifying cross-system interactions
- Role, permission, and access-control scenarios per persona
- Regression-sensitive scenarios targeting change-risk areas
- State transition and lifecycle scenarios covering workflow state machines

Output Format

Each generated test case includes:

- Test case ID and descriptive name
- Acceptance criteria reference linking back to the original story
- Given, When, Then steps in standard BDD format
- Test type classification (functional, negative, edge, integration, etc.)
- Priority level
- Pseudocode stub for automation implementation

Two-Phase Review Process

The Testing Agent uses a two-phase review workflow:

Phase 1 — Initial Review: After generation, each test case is presented for individual review. Users can approve, reject, or edit each scenario. Bulk approve-all and reject-all actions are available.

Phase 2 — Regeneration Review: Approved scenarios from Phase 1 can be regenerated to a target count, expanding coverage. The regenerated set goes through a final review before submission.

Automation Delegation

After final approval, the Testing Agent can delegate test implementation to GitHub Copilot cloud agent:

Approved test cases are packaged into GitHub Issues in the target automation repository. Each issue includes the BDD scenario, acceptance criteria, and pseudocode stub. GitHub Copilot cloud agent is assigned to implement the test using repository conventions, existing step definitions, and page objects.

The agent creates a branch, implements the Behave BDD test, runs it, and opens a pull request linked to the issue.

Automation status is tracked through the lifecycle: pending → issue created → in progress → PR open → completed or failed.

The Automation Dashboard provides real-time visibility into delegation status across all team test cases.

Additional Capabilities

API Registration and Playground: Teams can register APIs manually or import from Swagger/OpenAPI specs. The API Playground allows batch execution of registered APIs using Excel/CSV uploads with response validation.

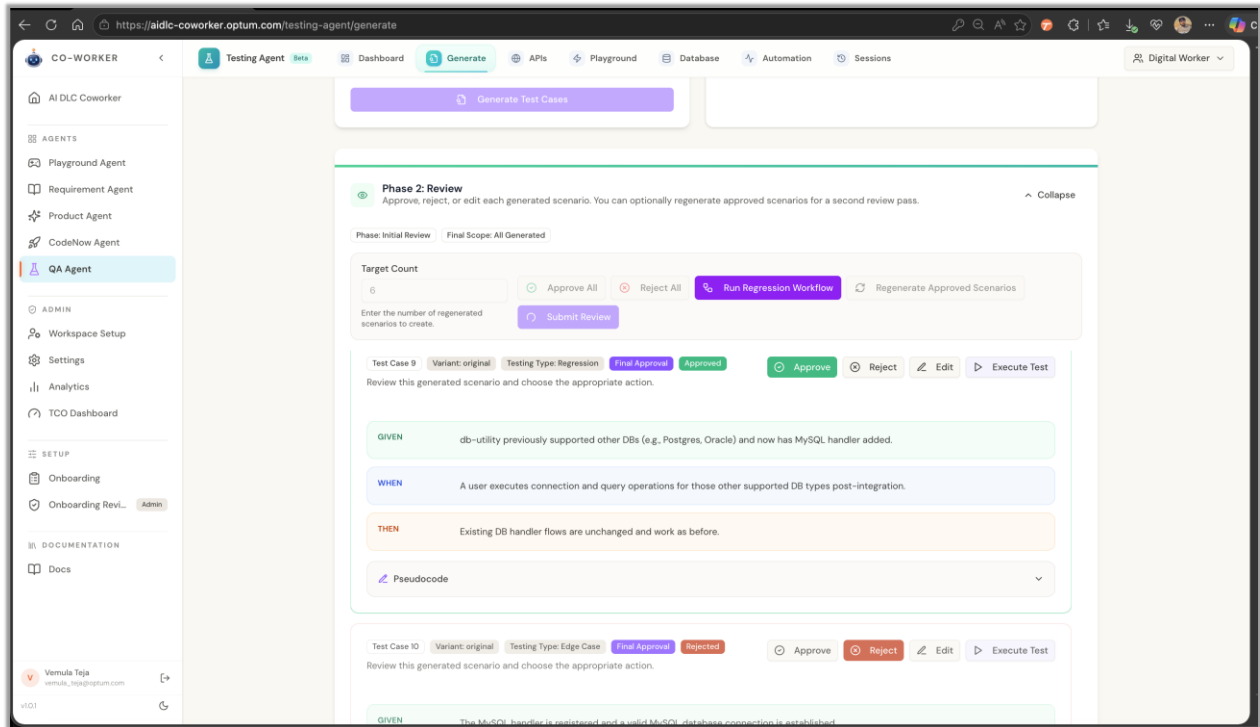
Database Setup: Teams can register database connections and predefined SQL queries for use in agent-driven test workflows.

Regression Workflow Execution: Pre-configured regression test workflows from Workflow Studio can be triggered directly from the Testing Agent with variable injection and execution tracking.

Session History: The Testing Agent displays available generation sessions. Session availability and retention may be subject to platform retention rules. If a displayed session cannot be resumed, create a new session and report the issue through the supported feedback channel.

Model Selection

The Testing Agent supports model profile selection. Teams can choose between the default model profile (GPT-4.1) for standard generation or the testing-optimized profile (GPT-5.1 mini) for faster iteration. Model selection is available at generation time.



Regression Suite

The Regression Suite allows teams to create, configure, and execute browser-automation-based regression workflows directly from the Testing Agent.

Create Your Own Regression Suite

Teams can build custom regression workflows from scratch:

- Provide a workflow name, description, step name, step task (AI agent instruction), and target application URL

- The system resolves the business hierarchy and creates a browser-automation workflow in Workflow Studio format

- Generate a Playwright/browser automation script by opening Chat2Prompt with the workflow context pre-loaded

- Finalize the workflow and track it from the Automation Dashboard

Execute Regression Workflows

Teams with a mapped regression workflow can execute it on demand:

Select your team and the system auto-loads the mapped regression workflow
Fill in required workflow variables (URLs, test data, environment parameters)
Fill in secret variables (encrypted credentials not shown in logs)
Optionally edit step prompts to modify AI agent task instructions per workflow step
Execute the workflow and the system generates parameters, uploads to the execution engine, and triggers the run
View results including job status (completed, failed, in progress), GIF recordings of browser sessions, test case validations with confidence scores, and event logs

Regression workflows use Chrome DevTools Protocol (CDP) for browser automation. All executions are tracked in transaction history for audit and review.

API Helper

The API helper provides API registration, testing, and validation capabilities within the Testing Agent.

API Registration

Teams can register APIs for testing and validation:

Manual Registration: Add APIs individually with method (GET/POST/PUT/DELETE), URL (supports variable placeholders), input parameters, headers, query parameters, request body configuration, and bearer token authentication.

Swagger/OpenAPI Import: Provide a Swagger or OpenAPI JSON URL to bulk-import all endpoints. The system parses paths, methods, and parameters automatically and registers all discovered APIs.

Each registered API includes typed input parameters (string, integer, boolean, date, file, float), configurable body format (JSON, XML, YAML, plain text), and optional response validation rules.

API Playground

The API Playground allows teams to execute registered APIs individually or in batch:

Select one or more registered APIs from the team API list

For individual testing: override headers, body, and query parameters, then execute and view the response (status code, response time, body, validation result)

For batch execution without parameters: run all selected APIs with their registered defaults

For batch execution with parameters: download a CSV/Excel template, fill in parameter values for each API, upload the file, and execute all APIs with the provided data

View results per API: success/error status, HTTP status code, response time, response body, and validation outcome

Export all results to CSV for reporting and audit

Response Validation

APIs can be configured with automated response validation:

Field Check: Validate specific JSON paths in the response against expected types, required flags, and expected values.

Query Match: Compare API response data against database query results using a registered database connection. Supports exact match, contains, and count comparison modes.

Combined: Apply both field checks and query match validation simultaneously for comprehensive coverage.

Database Helper

The Database Helper enables teams to register database connections and create reusable parameterized queries for use in test workflows and API response validation.

Database Connections

Register and manage database connections:

Add connections for Postgres, MySQL, SQL Server, Oracle, or SQLite

Provide a connection name and connection string (stored encrypted at rest)

Test connectivity before saving to verify the connection works

Edit or delete existing connections as needed

Connection strings are masked in the UI with a show/hide toggle for security.

Predefined Queries

Create reusable parameterized SQL queries linked to registered connections:

Provide a query name, select a database connection, choose a query type (CREATE, READ, UPDATE, DELETE), and write the SQL

Use ?param_name syntax in SQL to define parameters and the system auto-extracts them and creates parameter definitions

Define parameter types, descriptions, and constraints for each extracted parameter

Predefined queries are available for API response validation (query-match mode) and can be invoked by agent workflows during test execution

Database connections and queries are scoped per team. Queries with the databaseBased flag can be invoked directly by AI agent workflows during automated test execution.

20. Analytics Dashboard

The Analytics Dashboard gives teams and admins visibility into how the platform is being used and what it is costing across all agents.

What You Can See

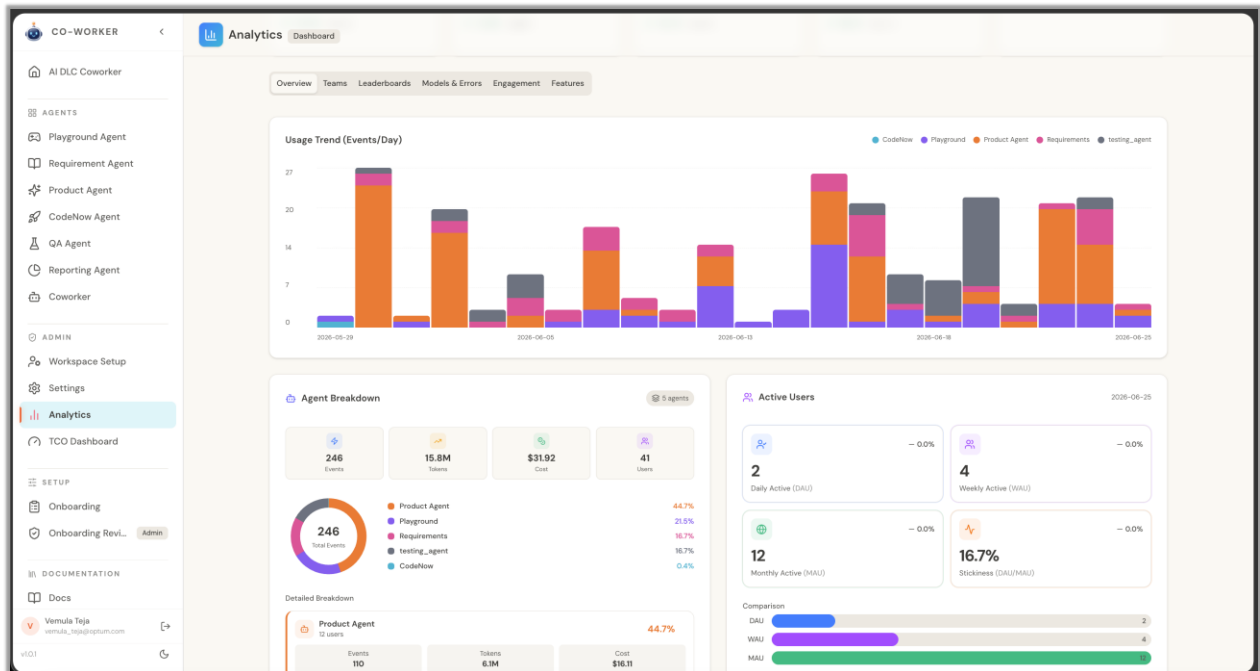
View	Description
Token Consumption	Input, output, and cached token usage broken down by agent, user, and time period
Cost Breakdown	Cost per agent, per feature, and per team
Active Users	Which team members are using the platform and how frequently
Request History	A log of agent invocations with timing and cost details
Agent Activity	Which agents are most active across the team

Analytics visibility depends on the user's authorized role and scope. Team members see data available to their team; broader views require the applicable authorization.

Responsible AI note: Usage analytics are intended to support platform operations, adoption, cost attribution, governance, and appropriate-use monitoring. Analytics must be used in accordance with applicable workforce, privacy, and data-governance requirements.

Cost Calculation

Costs are calculated based on LLM token usage using Azure OpenAI pricing with enterprise discounts. Rates are managed by the platform team. Future releases will allow teams to configure their own LLM credentials, shifting cost responsibility to the consuming team and enabling more granular attribution.



21. TCO Dashboard

The TCO (Total Cost of Ownership) Dashboard lets teams compare the cost and effort of delivery before and after adopting AI-DLC Coworker. It requires two inputs before the comparison view becomes meaningful.

Prerequisites

TCO Baseline (Required before pilot)

Before starting your pilot, an Editor must enter your team's traditional SDLC baseline. This captures how your team worked before AI-DLC and is the benchmark everything else is measured against.

You will enter:

- Number of team members involved in delivery
- Average hours per feature by size (small, medium, large)
- Average cost per feature by size
- Typical cycle time from feature start to delivery

This is entered manually on the onboarding dashboard under **TCO Baseline Setup**.

TCO Autonomous (Filled after pilot starts)

Once your pilot is running, the platform may capture available delivery telemetry and allow teams to supplement it with baseline or observed delivery data. The specific automated

and manually entered fields depend on the current configuration. Review data completeness before using TCO outputs.

This is configured under **TCO Autonomous Setup** on the onboarding dashboard.

How TCO Data Is Derived

- Automatically captured: only telemetry confirmed as available in the current platform configuration.
- Team-provided: traditional effort, cost, cycle-time baselines, and observed delivery data entered by authorized users.
- Calculated: comparisons produced when sufficient baseline and lifecycle data are available.
- Indicative estimates: cost avoidance or productivity comparisons that depend on stated assumptions.

What the TCO Dashboard Shows

Once both inputs are in place, the TCO Dashboard provides:

View	Description
Before vs After Comparison	Side-by-side view of traditional SDLC cost and cycle time versus AI-DLC cost and cycle time
Feature Cost Breakdown	Cost per feature including LLM token costs and human review time
Cycle Time Compression	How much faster features are moving through the lifecycle with AI-DLC
AI vs Human Time	Breakdown of time spent by agents versus time spent by humans on each feature
Cost Avoidance	Estimated savings compared to the traditional baseline

TCO outputs are estimates derived from available platform telemetry, team-provided baselines, and configured assumptions. Completeness and accuracy depend on the quality and coverage of those inputs. TCO results should not be presented as realized savings without appropriate validation.

22. Settings Overview

Settings availability depends on your access role.

Connected System Configuration

Used by Playground, Requirement Agent, and CodeNow to ground agent responses in your team's approved connected systems and working context.

- Editors can create and manage shared team configurations
- Viewers can view configuration metadata

Rally Configuration

Used for Rally feature creation, Product Agent flows, and requirement handoff.

- Teams can use the centralized platform Rally API key or provide their own if they already have one
- Teams must provide their Rally Project ID for configuration
- All team members can view configuration metadata
- Editors can create, update, delete, test, and set default Rally configurations

Git Repository Configuration

Used by CodeNow Delegations for Default teams.

- The platform displays a sample of 250 repositories to verify connectivity. You can access any repository you have permissions for
- All team members can view configured repositories
- Editors can add, bulk-add, update, and remove repositories

GitHub integration is configured per user under **Settings > Git Integration** and becomes available after the pilot is started.

Context and Graphify Configuration

Used to provide agents with background knowledge about your team's systems and codebase.

- Upload architecture documents, coding guidelines, spreadsheets, PDFs, images, and meeting transcripts
- Run Graphify on your repository to generate a knowledge graph
- Graphify is optional but recommended. It can be generated from existing repositories or supplemented with other documentation
- Editors manage shared context files and Graphify configuration
- Active context files are injected into agent sessions at runtime
 - Data rule:** Never upload Protected Data (PHI, PII, member/provider data) as context.

Prompt and Skill Configuration

Used to tune AI agent behavior across the team.

- Prompts control how each agent reasons, what it prioritizes, and how it structures output
- Skills define specialist subagent capabilities and toolsets

- Every prompt change creates a new version. **Full version history and rollback** is available
- Editors should manage these settings carefully as changes affect all team users

Responsible AI note: Prompt and skill changes affect **all team members' outputs**. Versioning and rollback exist to support governance and accountability. Do not introduce prompts that direct agents to process Protected Data or perform out-of-scope tasks.

23. Best Practices

Situation	Recommendation
Starting with the platform	Complete the full onboarding checklist before running agents. Context, Rally, and TCO Baseline setup all affect output quality and measurement
New to the product or system	Start with Playground Agent. Ask questions about your uploaded context and codebase before writing requirements or code
Requirements are unclear or scattered	Use Requirement Agent. Upload all available source material and let the agent consolidate and structure it
Setting up an initiative	Create a Pod of 3 to 4 members with clear roles. Round-robin assignment works best when Pod members are assignable and have notification preferences set
Need cross-agent traceability	Link notebooks and features to your initiative in Workspace Setup. Every agent action will be recorded in the feature timeline
Rally Feature needs breakdown	Use Product Agent before moving to implementation. Richer feature descriptions produce better story and task output
Default team needing implementation	Use CodeNow Delegations. Scope the task clearly and review the resulting pull request before merging
ServiceNow team needing implementation	Use CodeNow Studio. Provide explicit acceptance criteria, schema context, and target components for best results
Moving generated work into delivery	Use QA Agent to generate and review test scenarios before handoff
Reviewing any AI output	Always review, validate, and apply your own judgement. Agents produce reviewable drafts, not final decisions. You remain accountable for what you approve.
Handling data	Never upload or input PHI, PII, member/provider data, credentials, or other Protected Data into any agent.
Encountering wrong/biased/unsafe output	Reject it, do not use it, and report it via the feedback/escalation channels in Section 24.
Tracking value	Fill in TCO Baseline before the pilot starts and keep TCO Autonomous data current so the comparison view stays meaningful

24. Reporting Issues, Feedback & Escalation

Responsible AI depends on users actively flagging problems. If an agent produces output that is **inaccurate, biased, unsafe, non-compliant, or unexpected**, or if you suspect Protected Data was processed, take action immediately.

What to Do

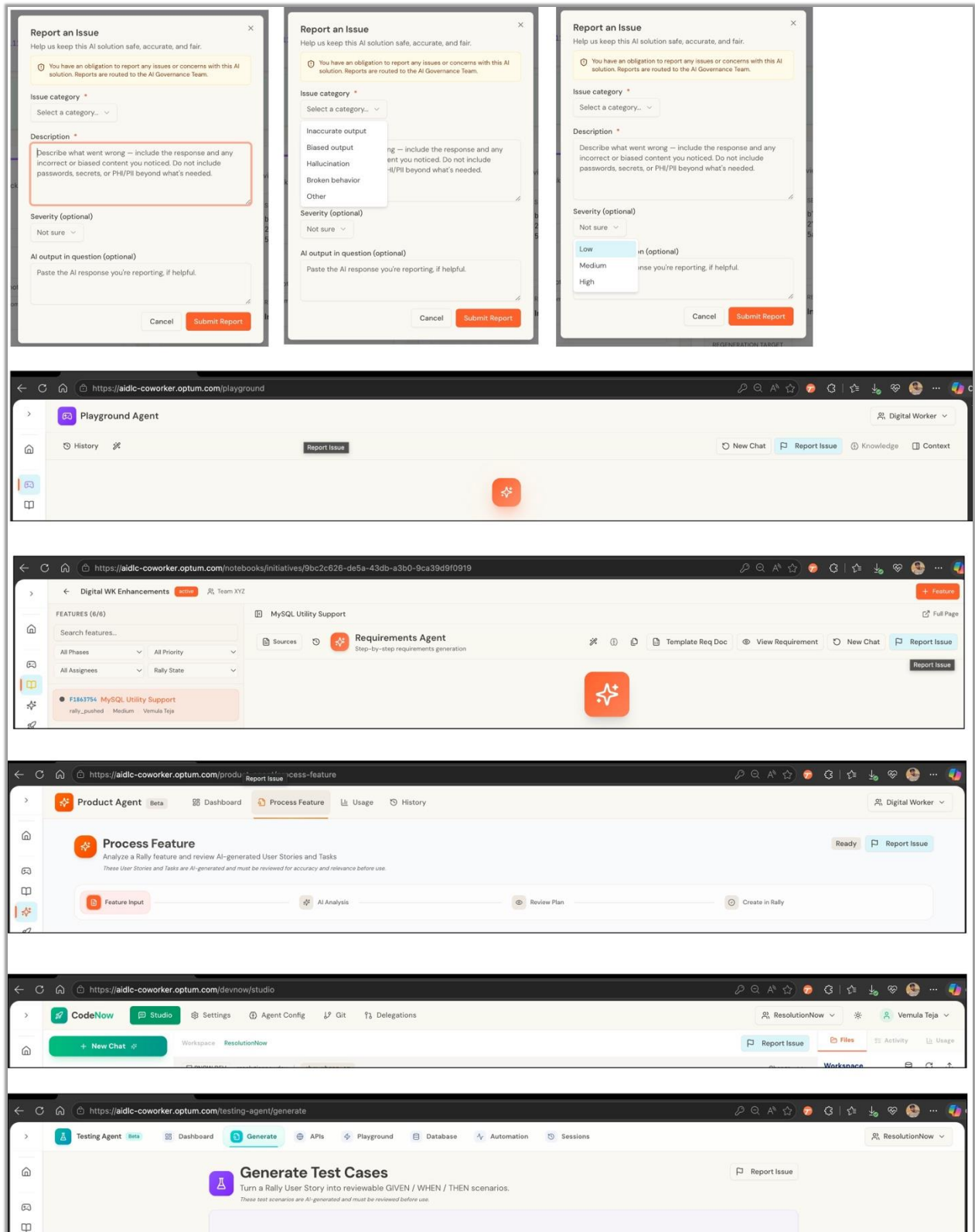
- **Do not use or propagate** the problematic output downstream.
- **Reject / discard** the output within the agent's review controls.
- **Report it** using the channels below with as much detail as possible (agent, prompt, output, expected behavior).

Where to Report

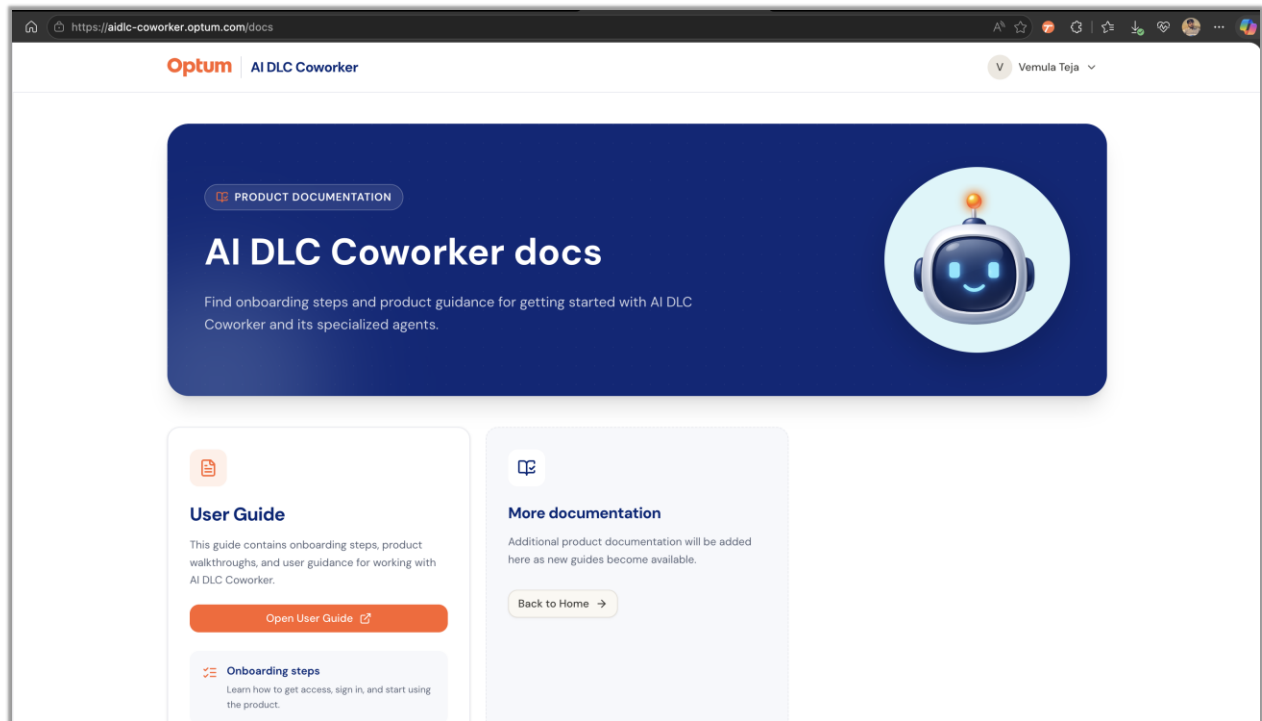
Issue Type	Contact / Channel
Inaccurate, biased, or unsafe AI output	User Report Issue button within the agents
Suspected Protected Data (PHI/PII) exposure	Follow standard incident reporting; notify the Platform Team and Privacy Office immediately
Access, onboarding, or technical issues	AI-DLC Coworker Platform Team <AgenticForce@ds.uhc.com>
Security concern	Follow standard enterprise security incident process

Your feedback improves the platform. Reported issues help the Platform Team refine prompts, skills, guardrails, and documentation.

In-app feedback / report mechanism: Report Issue button/form is available in all the agents



User guide - <https://aidlc-coworker.optum.com/docs>



25. Troubleshooting

I cannot sign in or I see an unauthorized page

Confirm that you are a member of either the Viewer or Editor secure group for your team. If your access was just approved, allow time for group membership to sync through SSO. Log out and sign back in after the sync.

I signed in but do not see my team in the Team Selector

Confirm you are in one of the team's secure groups:

```
AZU_COWORKER_<TEAM_NAME>_VIEWER  
AZU_COWORKER_<TEAM_NAME>_EDITOR
```

Example for a team named **ResolutionNow**:

```
AZU_COWORKER_RESOLUTIONNOW_VIEWER  
AZU_COWORKER_RESOLUTIONNOW_EDITOR
```

The Team Selector is populated at login. If your group membership was recently approved, log out and sign back in to refresh your session.

I can see the team but cannot edit settings or configurations

You likely have Viewer access. Request Editor access if you are expected to manage shared configurations, prompts, skills, Rally settings, Git repositories, model settings, or Orchestrator setup.

I cannot start the pilot

The Start Pilot action requires all four mandatory setup items to be complete: Workspace and Access, Rally Setup, Context and Graphify Setup, and TCO Baseline Setup. Check the onboarding dashboard for which items are still pending.

An agent says no connected-system configuration is available

Ask a team Editor to add or validate a shared connected-system configuration. Alternatively, create a personal configuration if that option is available for your role.

Rally actions are unavailable or features are not loading

Confirm that a Rally configuration has been set up by a team Editor. Ensure the correct Rally Project ID has been entered and the API key is valid. Editors can test the Rally configuration from the Settings page.

GitHub integration or Delegations are not available

GitHub integration is configured per user under Settings > Git Integration and is only available after the pilot has been started. Complete the onboarding checklist and launch the pilot first, then configure your personal GitHub token.

Delegations are not progressing

Confirm that:

- Your GitHub personal access token is configured under Settings > Git Integration
- The target repository is connected and accessible
- GitHub Copilot cloud agent has the required access to the repository
- The delegation task is clearly scoped. Vague prompts may result in incomplete or off-target output

Generated output looks off, biased, or is not relevant to my context

First, **do not use the output**. Then review the source context, uploaded documents, team configuration, prompts, and skills:

- Ensure relevant context documents are uploaded and active
- Consider running Graphify on your repository to improve codebase awareness

- For CodeNow Studio (ServiceNow teams): provide more explicit acceptance criteria, schema context, target components, and constraints
- For Delegations (Default teams): ensure the task is well scoped and the target repository and branch are correctly selected
- Editors can adjust prompts or skills to tune agent behavior for the team
- If the output is biased, unsafe, or non-compliant, **report it** (see Section 24)

The repository list in Git settings only shows 250 repositories

This is expected. The platform displays a sample of 250 repositories to verify connectivity. You can access any repository you have permissions for. The 250 shown is for verification purposes only, not a limit on what you can use.

TCO Dashboard shows no data

The TCO Dashboard requires both TCO Baseline and TCO Autonomous setup to be completed. If the Baseline has not been filled in, the before-and-after comparison will not have a reference point. Go to the onboarding dashboard and complete both TCO setup items.

27. Glossary

Term	Definition
Agent	A specialized AI capability within the platform (Requirement, Product, CodeNow, QA, Playground).
Advisory Draft	An AI-generated output that is a suggestion only and requires human review/approval before use.
Delegation	A scoped coding task handed off to the GitHub Copilot cloud agent that returns as a reviewable pull request.
Graphify	A utility that builds a knowledge graph of a codebase to give agents application-level understanding.
Human-in-the-Loop (HITL)	A control requiring a human to review and approve AI output before it is used downstream.
Initiative	A top-level container for a body of delivery work (project/epic).
LLM Gateway	The approved enterprise channel through which the platform accesses foundation models.
Orchestrator	The component that records initiative, feature, timeline, and human-decision events for traceability.
Pod	A small group (3–4) of team members responsible for reviewing agent outputs under an initiative.
Protected Data	PHI, PII, cardholder, biometric, and other regulated/sensitive data — prohibited in this platform.
TCO	Total Cost of Ownership — before/after delivery cost and effort comparison.

AI-DLC Coworker generates reviewable, advisory outputs only. It does not directly deploy to downstream systems, and it does not make autonomous decisions. All consequential

*actions require human review and approval. **You remain accountable for every output you accept, approve, merge, or release.***